II. Abstract

Long queues in the RTU canteen have consistently caused delays and inconvenience for both students and faculty, especially during peak hours. To address this problem, the RTUEats mobile application is proposed as an integrated food ordering and POS management system designed to eliminate physical lines and improve service efficiency. The system allows users to browse menus, place orders, and receive real-time in-app notifications for order updates, replacing the need for SMS or web alerts. RTUEats also introduces an in-campus delivery feature, where student couriers handle deliveries to designated campus areas—providing convenience for customers while offering part-time earning opportunities for students. By centralizing transactions, automating order flow, and removing on-site pickups, the system aims to minimize congestion, streamline canteen operations, and provide a faster and more organized ordering experience for the RTU community.

III. Introduction

Technological advancements have reshaped service industries worldwide, including the food service sector, where digital tools such as mobile ordering and integrated Point of Sale (POS) systems have become important for enhancing operational efficiency and customer satisfaction. Canteens in academic institutions, in particular, face substantial challenges during peak hours due to high customer densities, limited staff, and issues in manual ordering processing. These challenges

typically result in long queues, slow service, order inaccuracies, and overall reduced customer satisfaction.

According to Peblla (2024), mobile and integrated POS systems play a vital role in reducing waiting times and improving service efficiency by digitizing the ordering and payment workflow, thereby minimizing the reliance on manual processes. Digital ordering systems not only focus on customer interaction but also enhance operational accuracy. Research shows that online ordering platforms significantly reduce order errors by ensuring that customer choices are transmitted directly and clearly to the kitchen, eliminating issues associated with verbal and handwritten orders. Fonngo et al. (2020) reported that the adoption of a web-based canteen ordering and payment system in a university setting enhanced transaction speed and reduced manual workload for both customers and staff.

Given these technological advancements and the growing expectations of digital-native students, the need for an improved ordering and payment system in university canteens has become increasingly relevant. At Rizal Technological University (RTU), traditional ordering methods often cause congestion, delayed service, and miscommunication during peak hours, affecting both customer satisfaction and staff efficiency. As such, designing an integrated POD and mobile ordering system presents a timely and practical solution to enhance customer flow, streamline operations, and modernize the canteen’s service delivery. This study aims to explore the effectiveness of such a system, focusing on its potential to address the existing operational challenges within the RTU canteen.

Background of the Study

Traditional canteen operations commonly encounter issues such as long waiting times, overcrowding, and miscommunication in orders due to manual or verbal ordering methods. These problems not only disrupt customer flow but also reduce service quality and staff productivity. According to Peblla (2024), mobile POD technologies significantly improve service speed and reduce human error by automating the ordering and payment process. Similarly, studies on online food ordering systems highlight how digitalization increases order accuracy, enhances convenience, and allows real-time communication between customers and staff (GoTeso, 2023). Integrated systems also provided valuable data insights, such as real-time sales tracking and inventory monitoring, enabling more efficient management and informed decision-making (Hashmato, 2024).

A web-based canteen ordering system developed for a university proved successful in simplifying transactions and reducing manual tasks for both customer and staff (Fonggo et al., 2020). Moreover, smart canteen management studies emphasize the role of mobile ordering systems in reducing physical queues and creating a more organized dining environment (Auti et al., 2023). At Rizal Technological University (RTU), the canteen frequently experiences crowding and slow services during peak hours, leading to decreased customer satisfaction and inefficient workflow. Implementing an integrated POS and mobile ordering system offers a timely and practical approach to improving customer flow, optimizing staff workload, and aligning the canteen’s services with modern technological expectations.

Statement of the Problem

1. How can an integrated POS and ordering system improve the efficiency of customer flow in RTU canteen operations?
2. What system features are needed to ensure usability, reliability, and effectiveness for both customers and canteen staff?
3. How can a digital ordering system help reduce customer waiting time and prevent long queues during peak hours?

Objectives of the Project

General Objective

The primary objective of this study is to develop an integrated Point-of-Sale (POS) and mobile application ordering system intended to enhance customer flow, operational efficiency, and overall service quality within the RTU canteen.

Specific Objectives

Specifically, this study aims to:

- **Develop a mobile ordering platform** that enables users to view menu offerings, place orders, and receive timely updates on order status.
- **Integrate the mobile application with a centralized POS system** to streamline order processing, reduce manual handling, and minimize delays during peak service hours.

- **Establish an organized delivery** coordination process that helps manage customer traffic and prevents congestion at the canteen area.
- **Design a unified database system** capable of securely handling user information, orders, inventory data, and transaction records.
- **Evaluate the system's usability, accuracy, and overall performance** based on the experiences of students, faculty, and canteen personnel.

Significance of the Study

The development of an Integrated POS and Mobile Ordering System for the RTU canteen holds meaningful value for the institution, its students, and its operations. As RTU experiences growing student populations and increasingly busy peak hours, long queues, slow transactions, and service congestion continue to disrupt daily routines. This study provides a practical and timely response to these concerns by offering a system that streamlines ordering, minimizes face-to-face congestion, and strengthens overall service efficiency.

For Students, the system offers a more convenient and organized dining experience. Through real-time menu visibility, in-app order updates, and options for scheduled pick-ups or campus-based delivery, students can avoid long lines and better manage their limited break times. This helps reduce stress, promotes punctuality in classes, and ensures that students can access meals without unnecessary delays.

For Canteen Staff and Management, the integration of a digital POS supports faster and more accurate transactions by reducing manual errors in order handling and payment processing. Inventory monitoring becomes more systematic, allowing the canteen to anticipate demand, prevent shortages, and improve preparation efficiency. The system provides a centralized platform where orders, payments, and reports are organized, helping staff work more efficiently during peak periods.

For Student Couriers, particularly those participating in the delivery component, the system provides opportunities for part-time income while maintaining organized and traceable order assignments. It also builds practical skills in customer service, time management, and digital task coordination—skills that students can carry into future employment.

For the Institution, the project demonstrates RTU's commitment to modernizing campus services and embracing digital transformation. Implementing an innovative system such as RTUEats strengthens institutional efficiency, enhances campus quality of life, and supports the university's goals of fostering technologically capable graduates. The project also serves as a model for future campus-wide digital innovations.

In general, this study is important because it solves a long-standing operational problem with a solution that is both technically possible and meets the genuine demands of the RTU By improving service flow, reducing congestion, and enhancing user experience, the system contributes to a more organized, modern, and student-centered campus environment.

Scope and Limitations

Scope of the Study

The scope of this study is centered on the development and evaluation of an integrated Point-of-Sale (POS) and mobile application ordering system tailored for the operational needs of the Rizal Technological University (RTU) canteen. The system is designed to enhance service efficiency by providing a platform through which students and faculty can view available menu items, place orders, receive updates, and track the preparation status of their purchases. On the administrative side, the POS component consolidates incoming orders, facilitates transaction processing, manages basic inventory updates, and organizes order queues to support smoother canteen operations.

The research covers the system's design, programming, interface creation, and usability testing. It also assesses the system's efficacy in shortening client wait times, boosting transaction flow, and improving the overall ordering experience. Selected RTU students, faculty members, and canteen workers will serve as respondents for functionality and usability testing.

Limitations of the Study

The study is limited to a single RTU canteen branch and does not extend to other campus food vendors. Financial auditing features, advanced inventory analytics, and comprehensive security testing—such as penetration or stress testing—are excluded from the system's development. External payment gateways

and digital wallets outside those included in the prototype are not integrated. The system's operational effectiveness is assessed only during standard school days and does not account for unusually high-demand scenarios such as university fairs, major events, or emergency schedules.

Additionally, the study assumes access to stable internet connectivity or campus Wi-Fi as a prerequisite for the mobile application's use. Broader management concerns, such as supplier coordination, staffing decisions, pricing strategies, and canteen policy-making, fall outside the study's coverage and are not examined.

IV. Review of Related Literature

Existing Campus Dining Systems

Traditional campus dining models have long relied on meal-plan programs and campus ID card systems to manage transactions and access to dining facilities. These systems are efficient for centralized meal plans but often lack flexibility for on-demand ordering and real-time order management; as a result, students still face long queues and constrained options during peak hours (Mowreader, 2024). Recent surveys and industry reports indicate growing student demand for touchless payments, mobile ordering, and app-based conveniences that allow users to pre-order, pay, and receive status updates — features that reduce perceived waiting time and better fit students' limited break periods (Society for Hospitality and Foodservice Management, 2025).

Several campuses have therefore begun supplementing traditional meal-card systems with mobile or campus-specific ordering platforms that integrate with existing student credential systems. These hybrid approaches preserve meal-plan accounting while enabling advanced ordering and in-app notifications, yielding improved throughput without fully replacing institutional payment infrastructures (Touchnet, 2024). Adopting such approaches demonstrates that campus dining systems can modernize incrementally—linking mobile interfaces to campus credentials and POS back-ends—to reduce congestion while maintaining institutional controls (Hudson, 2022).

(Mowreader, 2024; Society for Hospitality and Foodservice Management, 2025; Touchnet, 2024; Hudson, 2022.)

Industry Practices and Case Studies

Commercial food-service and campus deployments illustrate practical models for integrating POS, ordering, and fulfillment. Large vendors and chains have adopted cloud-based POS solutions and mobile ordering features that synchronize orders across channels, automate payment processing, and support scalable reporting (Zhong, Oh, & Moon, 2020). Case studies from campus implementations—such as partnerships between universities and delivery/ordering platforms—demonstrate how APIs and centralized POS systems can aggregate orders from kiosks, web apps, and third-party marketplaces into a unified workflow for kitchen staff and managers (Hudson, 2022).

Best-practice guidance from vendor platforms emphasizes secure integration of campus ID, payment, and POS systems to support multiple fulfillment modes (dine-in, grab-and-go, delivery) without fragmenting accounting and meal-plan reconciliation (Touchnet, 2024). Emerging logistics approaches—smart lockers, scheduled delivery windows, and student courier programs—also reveal how campuses can expand service reach while controlling staffing and space constraints, making these industry practices directly relevant to RTUEats' campus-delivery model (Campus ID News, 2022; Society for Hospitality and Foodservice Management, 2025).

(Zhong et al., 2020; Hudson, 2022; Touchnet, 2024; Campus ID News, 2022; Society for Hospitality and Foodservice Management, 2025.)

Academic Research on Campus Ordering Systems

Scholarly research supports mobile ordering and integrated POS as effective interventions for reducing queues and improving perceived service quality in campus environments. Empirical studies find that pre-ordering and scheduled fulfillment significantly lower physical queue length and waiting time, while real-time status updates reduce uncertainty for users (Prajapati & Shah, 2022; Hwang & Kim, 2021).

Evaluations of campus-focused prototypes and trials indicate that students value delivery and app-based ordering for convenience and time savings, particularly

when systems provide reliable menu availability and secure payment options (Hamid, Adnan, & Ekhsan, 2022).

System-design research also underscores technical choices—such as modular back-end architectures and relational DBMS usage—that support concurrency, transactional integrity, and reliable inventory tracking in busy food-service contexts. Studies that implemented MySQL- or equivalent-backed prototypes reported improvements in order accuracy and preparation coordination, reinforcing the practicality of database-driven POS + app solutions for institutional canteens (Dinesh & Thakare, 2025; Tan & Lee, 2023). These academic findings corroborate industry evidence and provide methodological support for a DBMS-centered approach in RTUEats.
(Prajapati & Shah, 2022; Hwang & Kim, 2021; Hamid et al., 2022; Dinesh & Thakare, 2025; Tan & Lee, 2023.)

Rationale for a Database-Backed System

A centralized DBMS is essential for managing orders, payments, inventory, and courier assignments in real time. Literature and cloud architecture standards emphasize that a robust database layer enables multiple clients—such as POS terminals, mobile applications, and kitchen displays—to perform simultaneous transactions while preserving consistency, durability, and accuracy, especially during peak operating hours (Amazon Web Services, n.d.; Touchnet, 2024). Implementing a relational DBMS, or a well-structured NoSQL alternative where appropriate, ensures

atomic order-payment processes and prevents issues such as partial updates, lost transactions, or inventory discrepancies.

Studies further highlight that database-driven systems support immediate inventory synchronization, reliable reporting, and resilience during intermittent connectivity (Prajapati & Shah, 2022; Dinesh & Thakare, 2025). For RTUEats, where large volumes of students may place concurrent orders and couriers depend on timely dispatch information, the DBMS serves as the authoritative source of truth. It enables seamless coordination between the mobile app, POS terminals, kitchen systems, and delivery assignments while ensuring scalability and supporting future feature expansion.
(Amazon Web Services, n.d.; Touchnet, 2024; Prajapati & Shah, 2022; Dinesh & Thakare, 2025.)

V. System Analysis and Design

Requirements Analysis (Functional and Non-Functional Requirements)

Functional Requirements

These define the specific behaviors and functions of the system.

Authentication & Authorization:

Users can register as a Student, Professor, Staff, or Courier.

Users can log in using email and password with role-based redirection.

Users can reset forgotten passwords via email OTP verification.

Admins can create, edit, and delete users directly from a dashboard.

Customer Module (Students/Professors):

View a list of available food products filtered by store status (open/closed) and target audience.

Search for products by name.

Add items to a shopping cart, adjust quantities, and remove items.

Place orders with specific delivery details (Building, Floor, Room) and payment methods (COD/GCash).

Track real-time order status (Pending, On Delivery, Completed).

Chat with the assigned courier, including sending image attachments.

View transaction history and save digital receipts as images.

Staff/Vendor Module:

Register a store name upon first login.

Toggle store status (Online/Offline) to control product visibility.

Add, edit, and delete menu items, including uploading product images.

View incoming orders and see courier assignment status.

Courier Module:

View a list of available "Pending" orders.

Accept jobs, which updates the order status to "On Delivery".

View delivery details (Pickup location, Drop-off location, Earnings).

Mark orders as "Delivered" to complete the transaction.

Non-Functional Requirements

These define the system's operational capabilities.

Real-time Performance: The system must push updates (chats, order status changes) instantly without manual refreshing using Supabase Realtime streams.

Data Integrity: Orders link specifically to users and products; inventory/menu items are linked to specific store owners.

Usability: The interface uses a consistent design language (Poppins font, rounded corners, specific color palette) for ease of use.

Reliability: The app includes a connectivity wrapper to alert users if the internet connection is lost.

Security: Passwords are handled via Supabase Auth; sensitive actions (like admin deletion) require specific role checks.

Database Design (ERD, Relational Schema, Normalization Process)

Entities & Attributes (Schema)

1. **profiles**

   Purpose: Stores user details extending the basic Auth table.

   Columns: id (PK), full_name, email, role, avatar_url, store_name (nullable), is_open (boolean).

2. **products**

   Purpose: Stores menu items.

   Columns: id (PK), name, price, description, target_audience, image_url, owner_id (FK to profiles.id).

3. **orders**

   Purpose: Stores the high-level transaction details.

   Columns: id (PK), user_id (FK), courier_id (FK), store_name, customer_name, courier_name, total_amount, status (pending/on_delivery/completed/cancelled), delivery_location, payment_method, created_at, items_summary (Denormalized string).

4. **order_items**

   Purpose: Stores specific items within an order (handling the Many-to-Many relationship between Orders and Products).

Columns: order_id (FK), product_id (FK), product_name, quantity, price_at_purchase, store_name, customer_name.

5. **chat_messages**

   Purpose: Stores communication between customer and courier.

   Columns: id (PK), order_id (FK), sender_id (FK), message, image_url, is_read, created_at.

Normalization

- **1NF (First Normal Form):** All tables have primary keys (id). Most columns are atomic. *Note:* The items_summary in the orders table is a comma-separated string, technically violating 1NF for that specific column, but order_items exists to maintain 1NF/2NF for data integrity while items_summary is likely used for display performance.
- **2NF (Second Normal Form):** All non-key attributes are dependent on the primary key. For example, product price and description depend entirely on product_id.
- **3NF (Third Normal Form):** Transitive dependencies are minimized. For example, product data is not repeated in the profiles table; it is separated into the products table and linked via owner_id.

System Architecture (Overview of the system module, hardware/software requirements)

System Modules

1. **Auth Module**: Handles Login, Signup, Session Management, and Password Recovery.
2. **Client Module (Customer):** Product browsing, cart management, and ordering.
3. **Vendor Module (Staff):** Product CRUD and Store status management.
4. **Logistics Module (Courier):** Job acceptance and delivery workflow.
5. **Admin Module:** User management and oversight.
6. **Communication Module:** Chat system with image support.

Hardware/Software Requirements

Software:

Frontend: Flutter Framework (Dart language).

Backend: Supabase (PostgreSQL, GoTrue for Auth, Realtime Engine).

IDE: VS Code or Android Studio (implied by Flutter project structure).

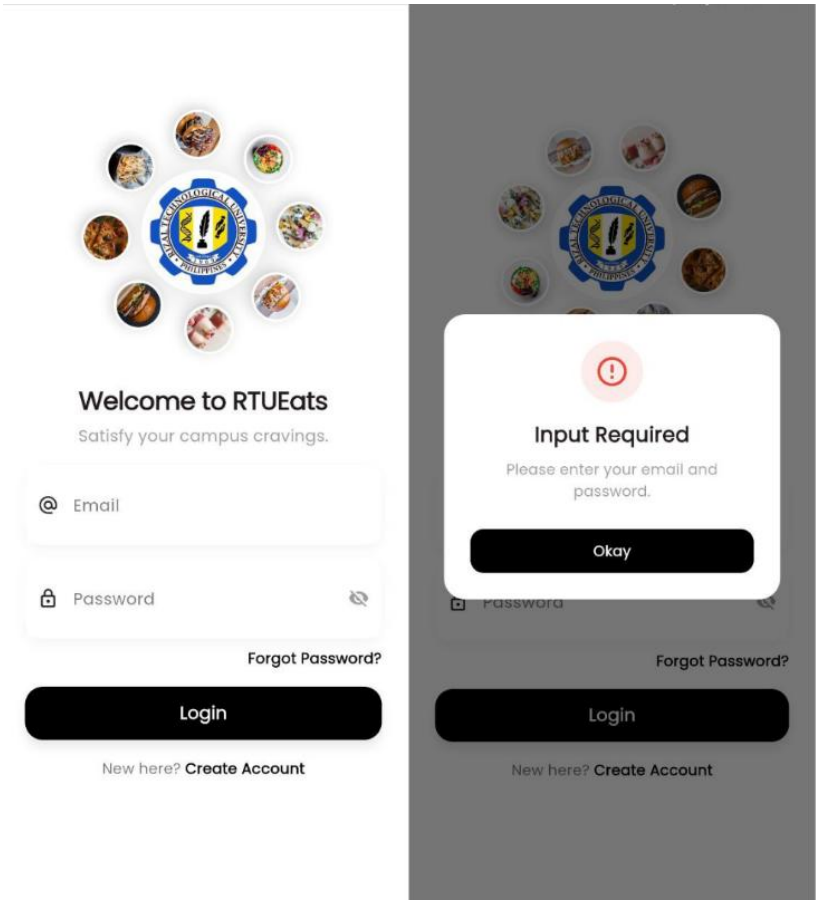
Hardware (User):

Mobile Device (Android/iOS) with internet connectivity.

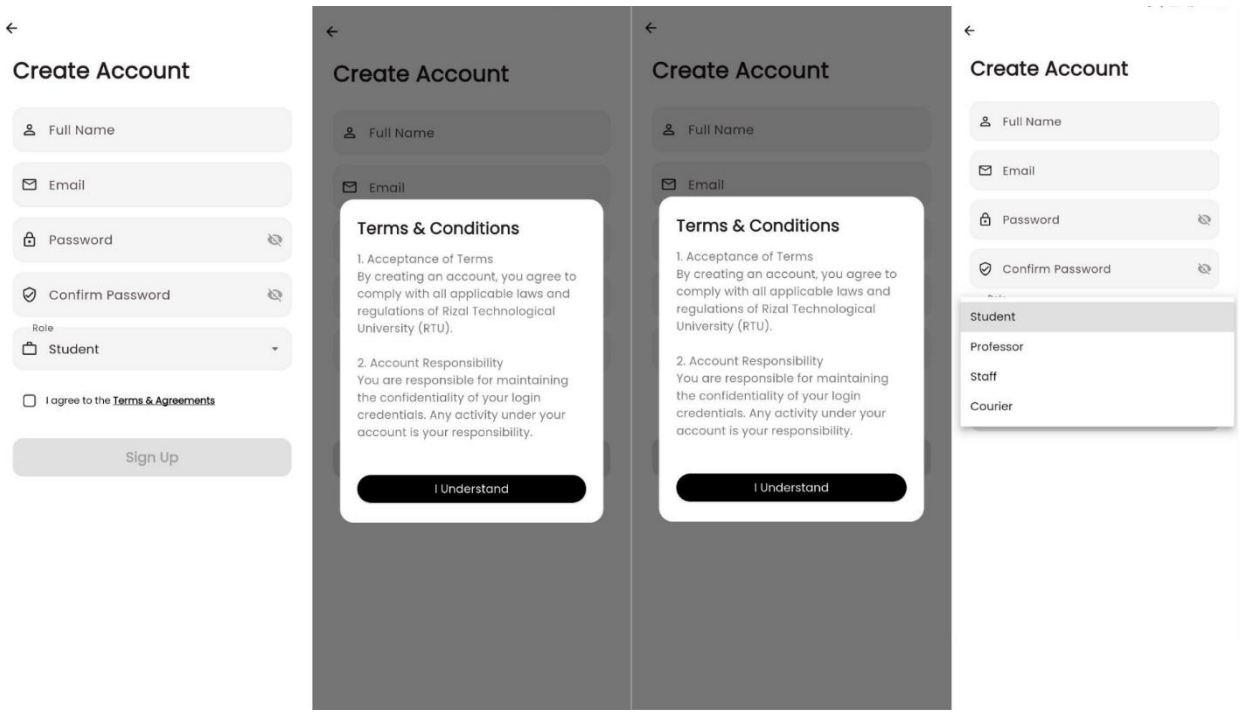
Camera (for uploading profile/product pictures).

Interface Design (User interface mockups/screenshots)

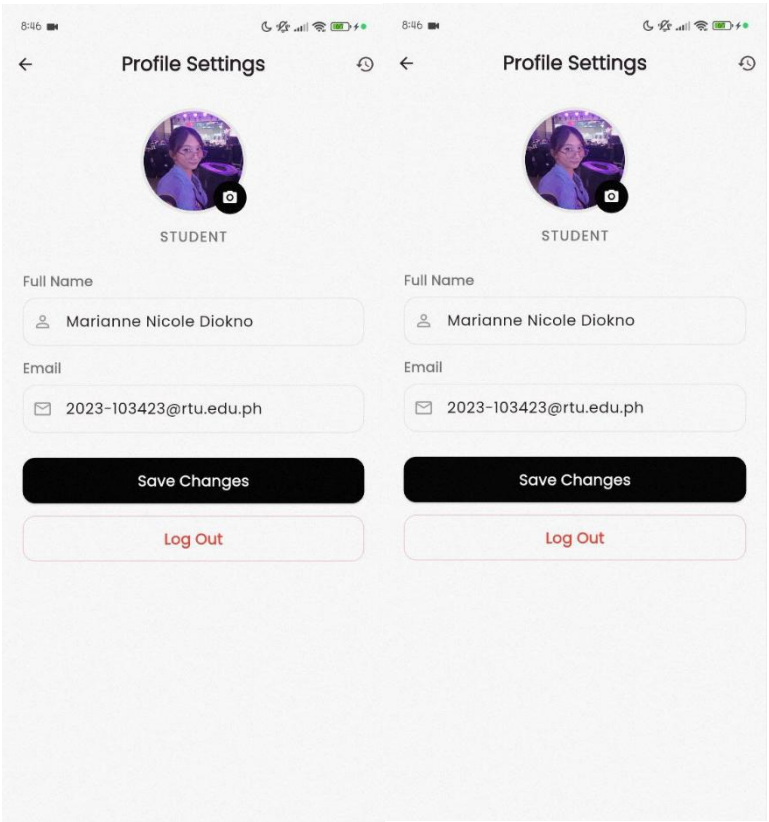
Sign Up Page



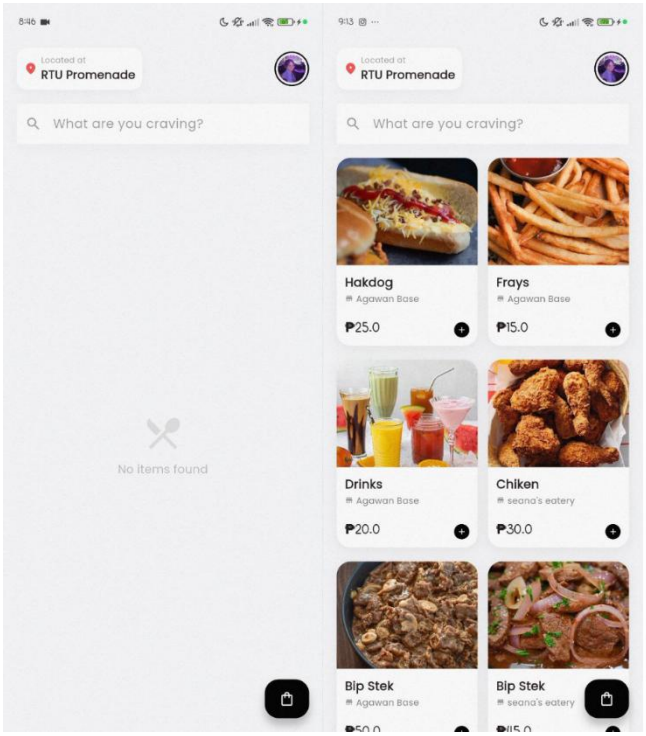
Create Account



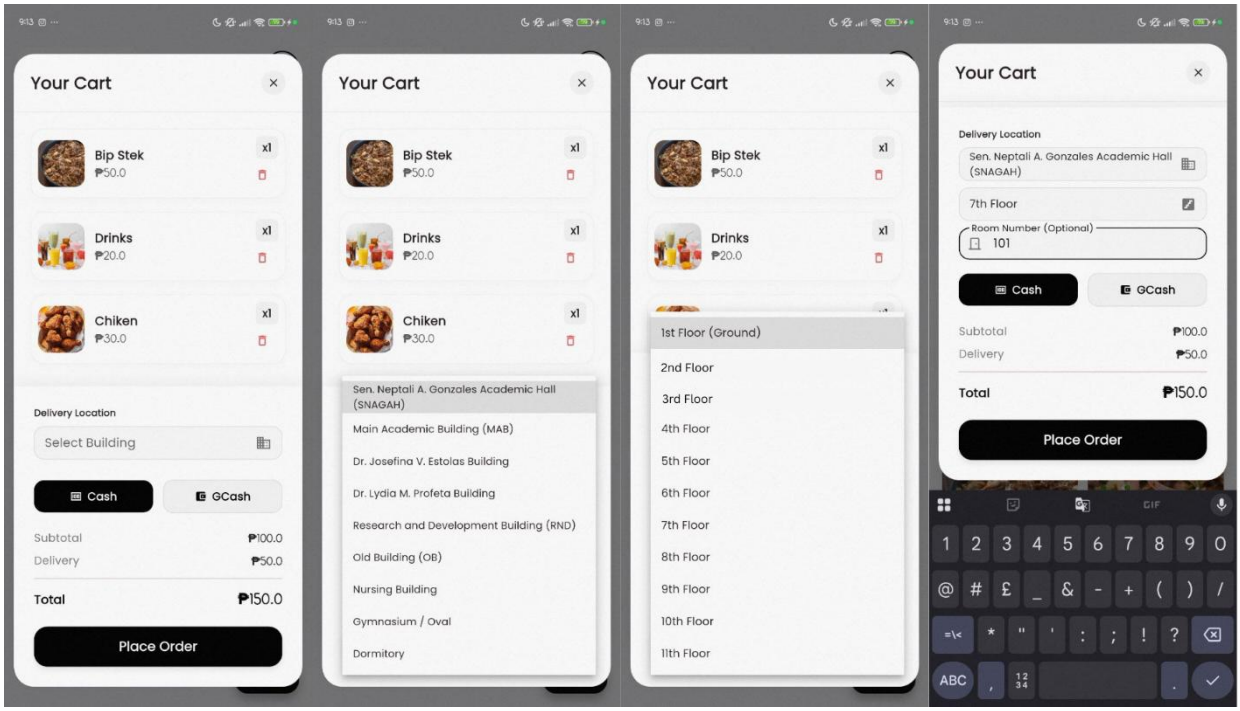
Profile Settings (Student)



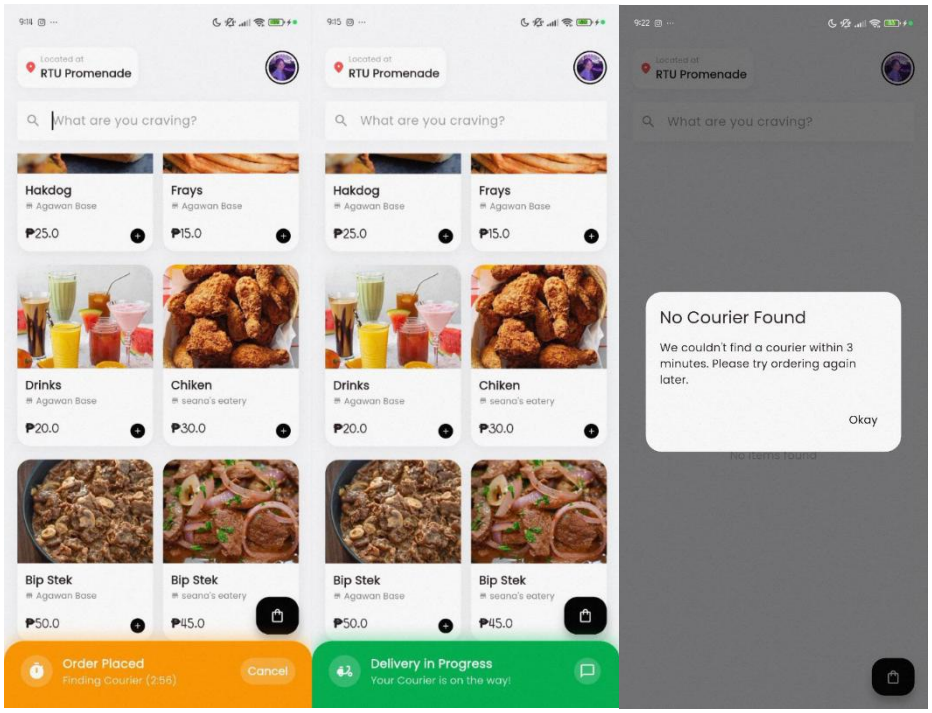
Student Dashboard



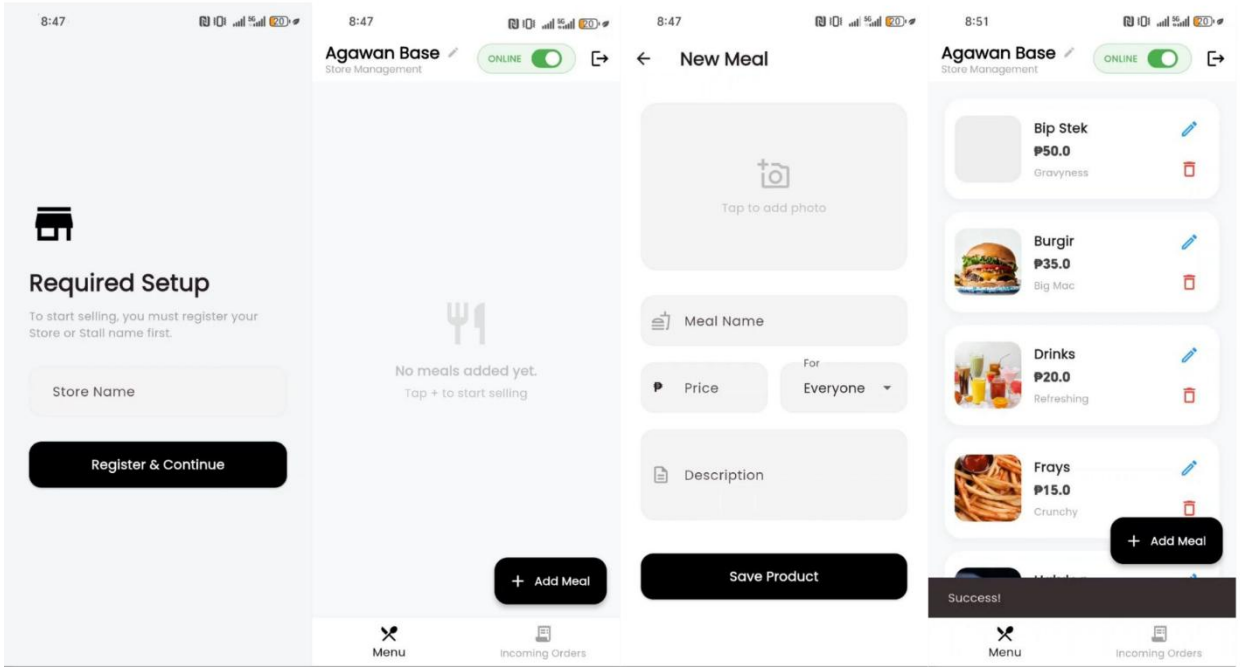
Ordering System / Add to Cart Feature



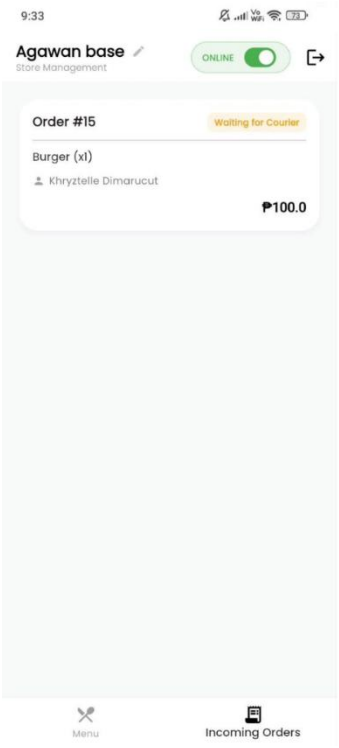
Finding Courier / Delivery in Progress / No Courier Found



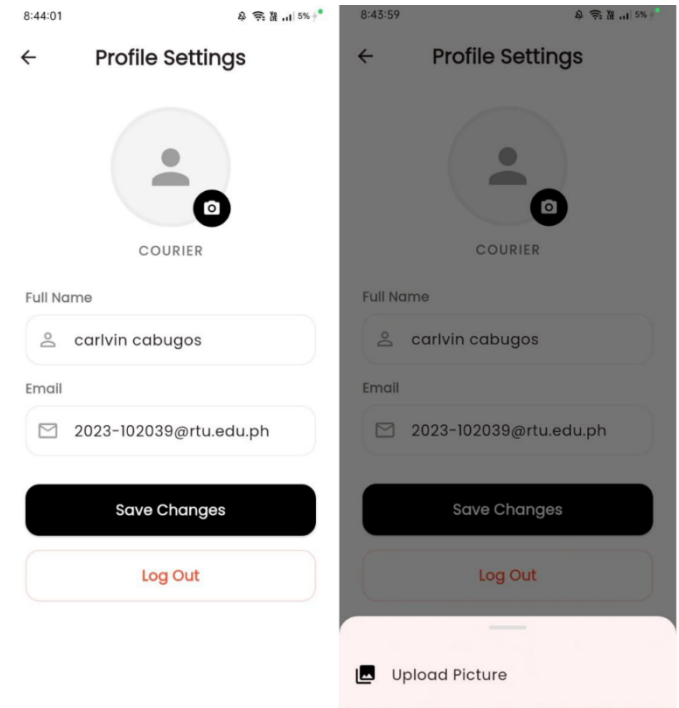
Staff



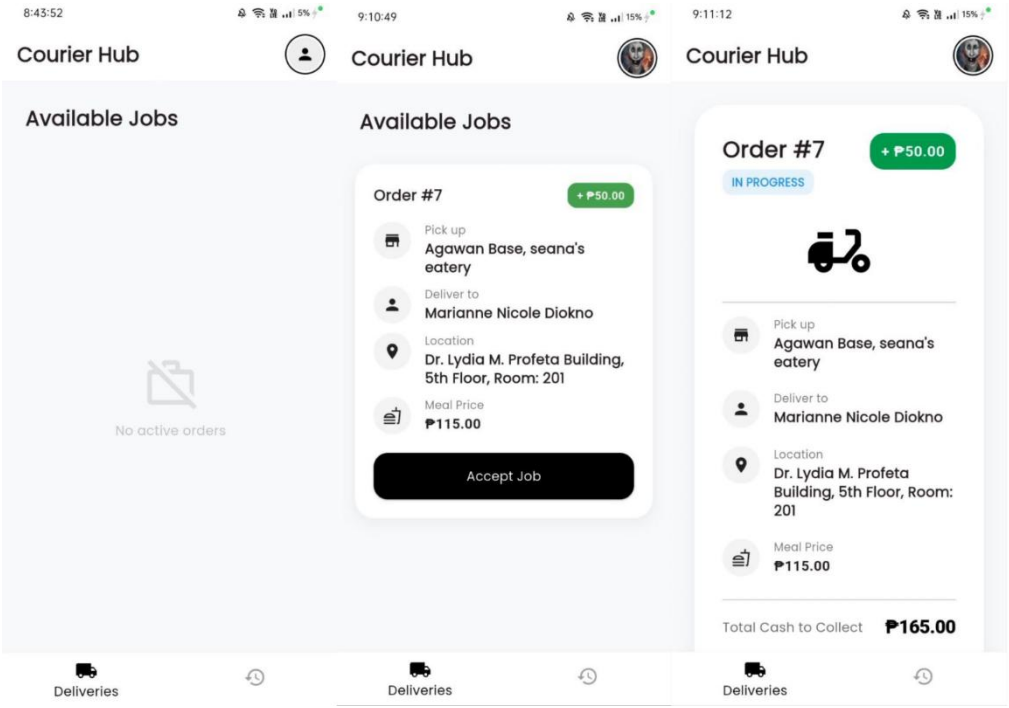
Staff (Incoming Order)



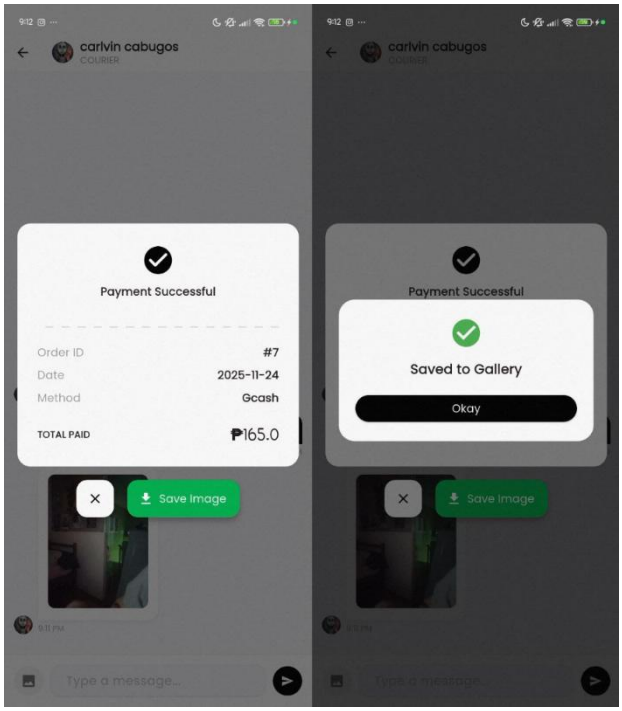
Courier / Delivery (Profile Settings)



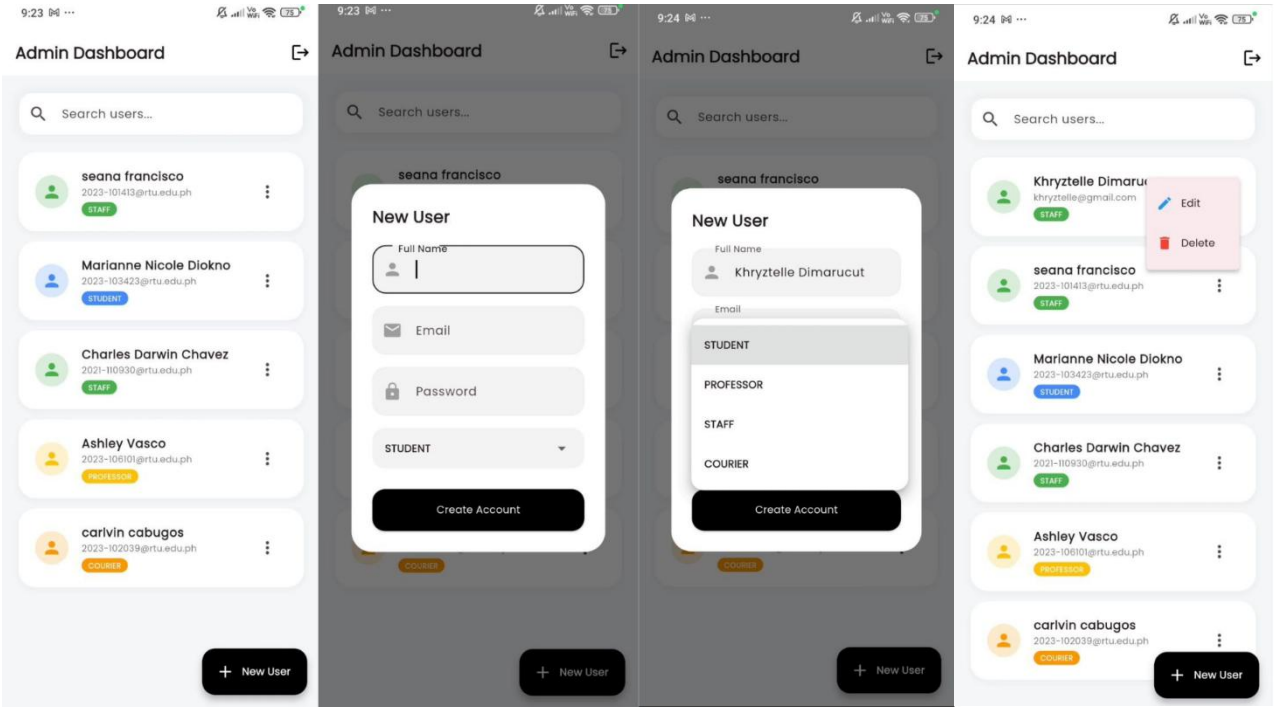
Courier Dashboard / In Progress Order

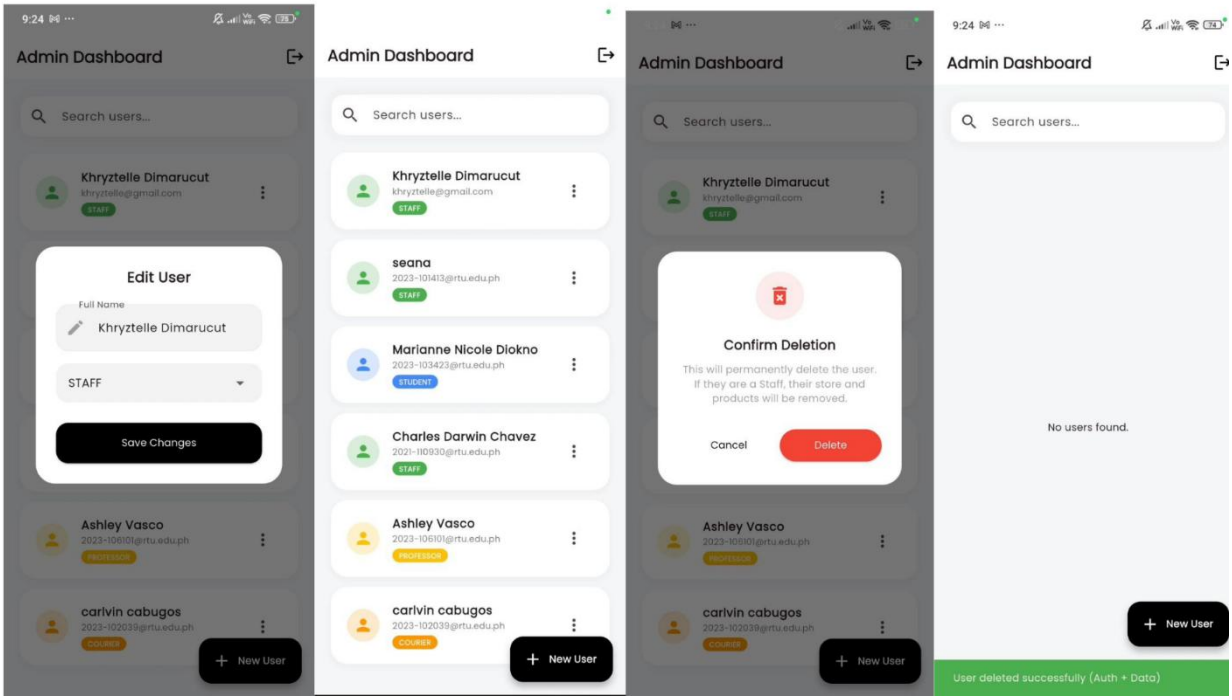


Message / Delivery Successful

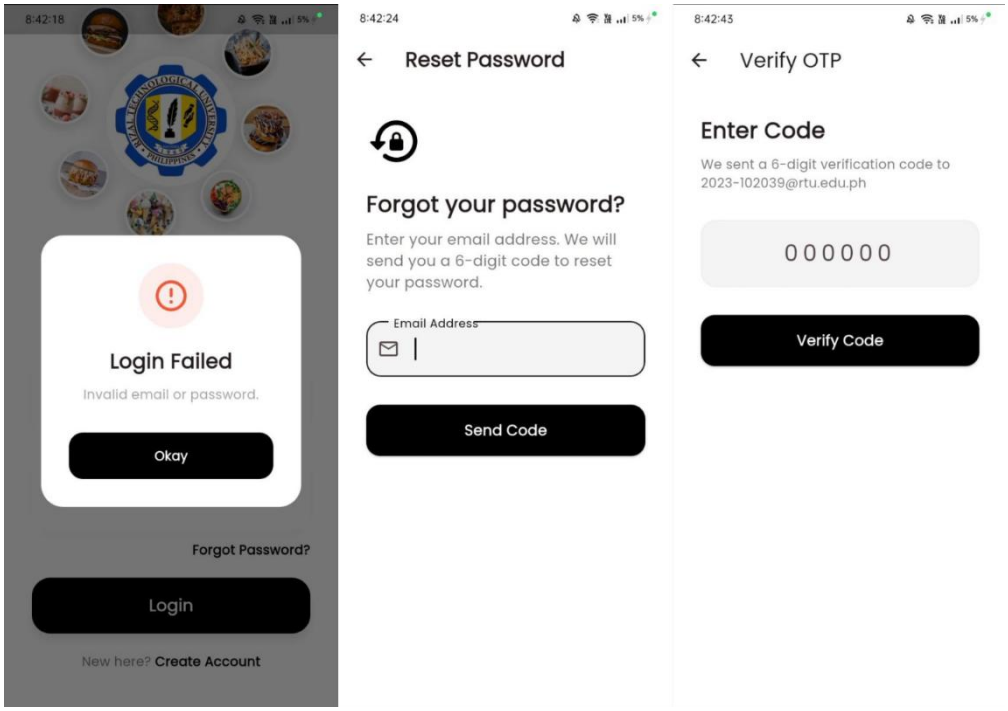


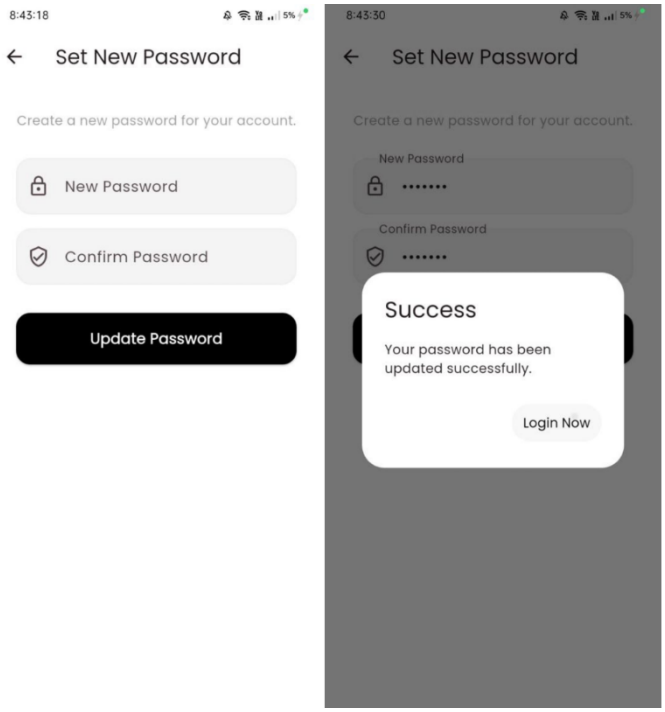
Admin Dashboard / Create, Read, Update and Delete





Forgot Password





Navigation flow

Initialization:

App Start -> ConnectivityWrapper (Checks Internet) -> AuthGate (Checks Session).

Authentication Flow:

If No Session -> LoginPage.

If LoginPage -> "Create Account" -> SignUpPage.

If LoginPage -> "Forgot Password" -> ForgotPasswordPage -> OtpVerificationPage -> NewPasswordPage.

Post-Login Routing (AuthGate Logic):

If Role = Admin -> AdminDashboard.

If Role = Courier -> CourierDashboard.

If Role = Staff -> Check store_name.

- If store_name is null -> StoreRegistrationPage.
- Else -> StaffDashboard.

If Role = Student/Professor -> CustomerHome.

Customer Workflow:

CustomerHome -> Tap Product -> ProductDetailsModal -> Add to Cart.

CustomerHome -> Tap Cart FAB -> CartScreen -> Checkout.

After Checkout -> CustomerHome (Bottom Sheet Status Bar appears) -> Tap Chat Icon -> ChatScreen.

On Completion -> ReceiptScreen (Dialog).

Staff Workflow:

StaffDashboard -> "Add Meal" FAB -> StaffAddProductPage -> Save -> Return to Dashboard.

StaffDashboard -> Edit Icon (on product) -> StaffAddProductPage (in edit mode).

VI. System Implementation

Tools and Technologies Used (DBMS, Programming Languages, Frameworks)

Frontend Development

Framework: Flutter (Mobile Application Framework).

Language: Dart.

State Management: Provider (Used for managing cart state and app-wide data).

Backend & Database (BaaS)

Platform: Supabase (Open-source Firebase alternative).

Database Engine: PostgreSQL (Underlying database for Supabase).

Authentication: Supabase Auth (Email/Password and OTP verification).

Storage: Supabase Storage (For images).

Real-time: Supabase Realtime (Used for chat and order tracking streams).

Key Packages & Libraries

supabase_flutter: For backend connectivity, auth, and database interactions.

image_picker: For selecting images from the gallery for profiles and product listings.

screenshot: For capturing the receipt widget as an image.

image_gallery_saver_plus: For saving the captured receipt to the device gallery.

permission_handler: For managing storage permissions.

Implementation Details (table structures, queries, stored procedures, triggers)

A. Database Schema (Inferred)

The application interacts with the following tables based on the Supabase SDK calls found in the code:

profiles (User Data)

Primary Key: id (Linked to Auth ID).

Columns: full_name, email, role (student, professor, staff, courier), avatar_url, store_name, is_open.

products (Menu Items)

Primary Key: id.

Columns: name, price, description, target_audience, image_url, owner_id (Foreign Key to profiles).

orders (Transaction Data)

Primary Key: id.

Columns: user_id, customer_name, store_name, courier_id, courier_name, items_summary, total_amount, status (pending, on_delivery, completed, cancelled), type, payment_method, delivery_location, created_at.

order_items (Order Details)

Columns: order_id, product_id, product_name, quantity, price_at_purchase, store_name, customer_name.

chat_messages (Messaging)

Columns: order_id, sender_id, message, image_url, is_read, created_at. admins (Access Control)

Columns: id (Checks if a user is an admin upon login).

B. Storage Buckets

The application utilizes Supabase Storage for handling media assets:

1. food-images: Stores profile avatars and food product images.
2. chat-attachments: Stores images sent within the chat feature.

C. Stored Procedures (RPC)

delete_user_complete: A Remote Procedure Call (RPC) is used in the Admin Dashboard. This suggests a PostgreSQL function exists on the backend to

handle the cascade deletion of a user from both the auth.users table and the public profiles table/related data.

D. Triggers and Real-time Logic

Streams: The app relies heavily on .stream() to listen for database changes in real-time.

Courier Dashboard: Listens to orders where the status is pending to show available jobs instantly.

Chat Screen: Listens to chat_messages ordered by created_at to update the conversation view dynamically.

Customer Home: Listens to orders to track order status (e.g., finding a courier, delivery progress).

E. Query Implementation

The application uses the Supabase Flutter SDK's method chaining for queries:

Filtering: Uses .eq() (equals), .neq() (not equals), and .maybeSingle() (returns null if no data found) extensively.

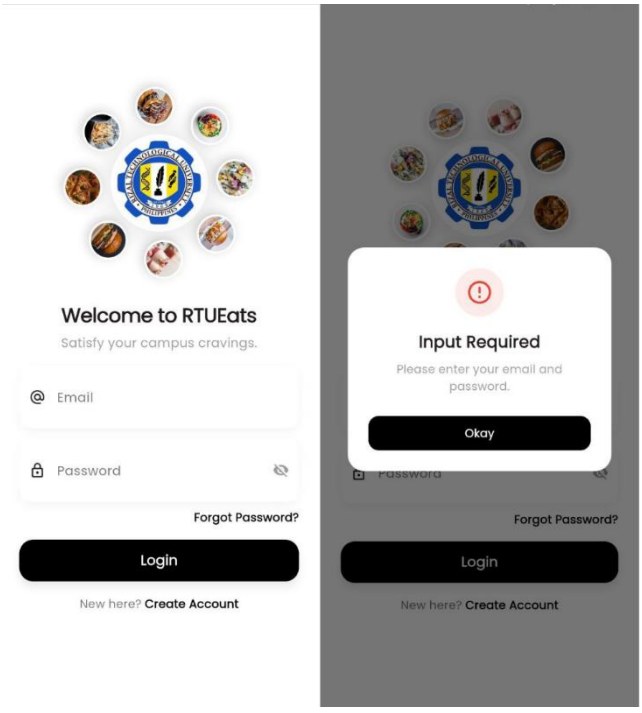
Example: Fetching an active order for a courier: .from('orders').select().eq('courier_id', userId).neq('status', 'completed').

Ordering: Uses .order('created_at', ascending: false) to show the newest items/chats first.

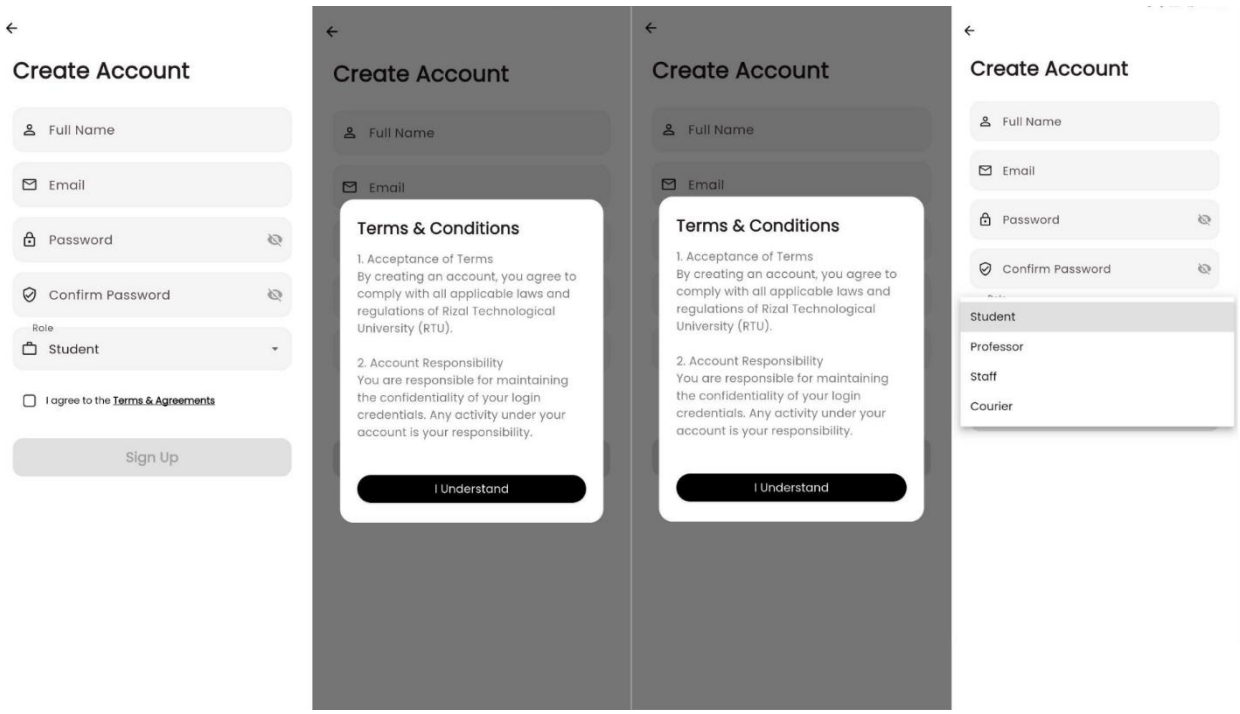
Upserting: Uses .upsert() when creating new admin-generated users to handle both insert and update logic efficiently.

Sample Screenshots of Working System

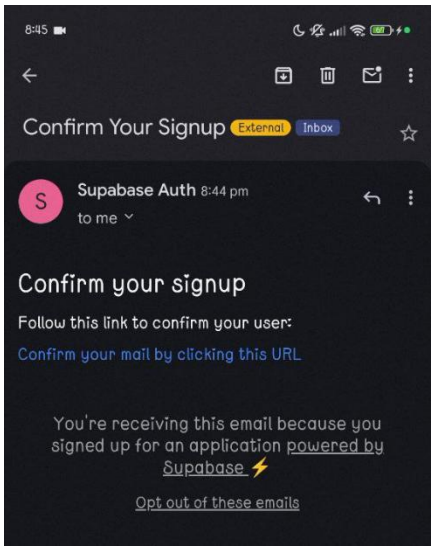
Sign Up Page



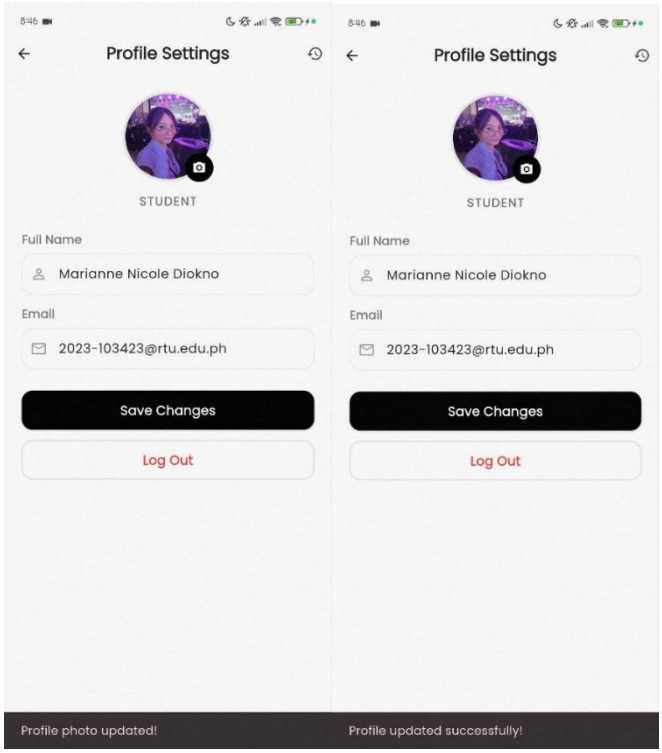
Create Account



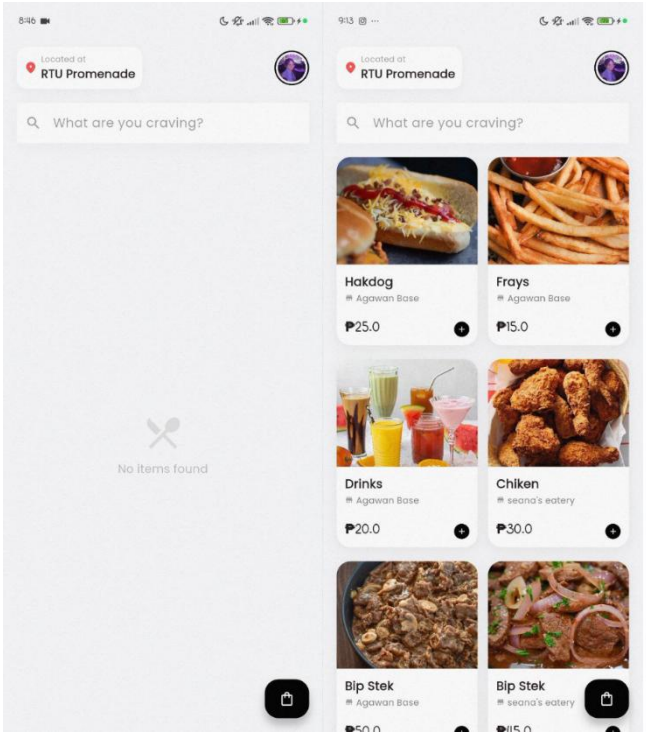
Create Account (Email Confirmation)



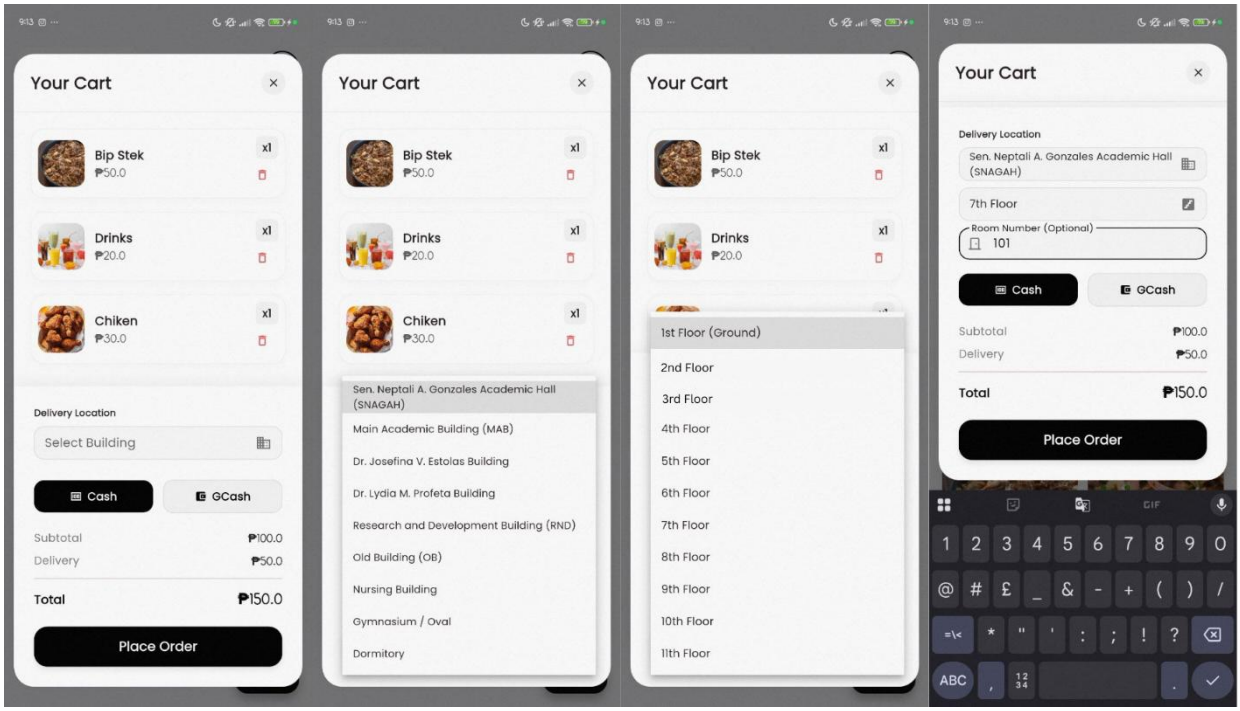
Profile Settings (Student)



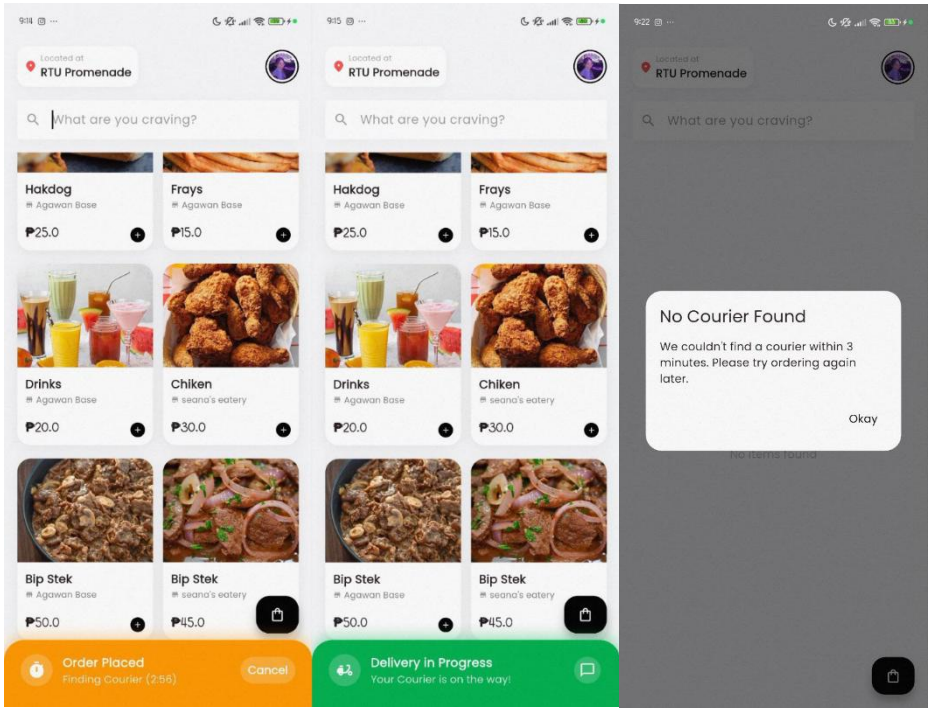
Student Dashboard



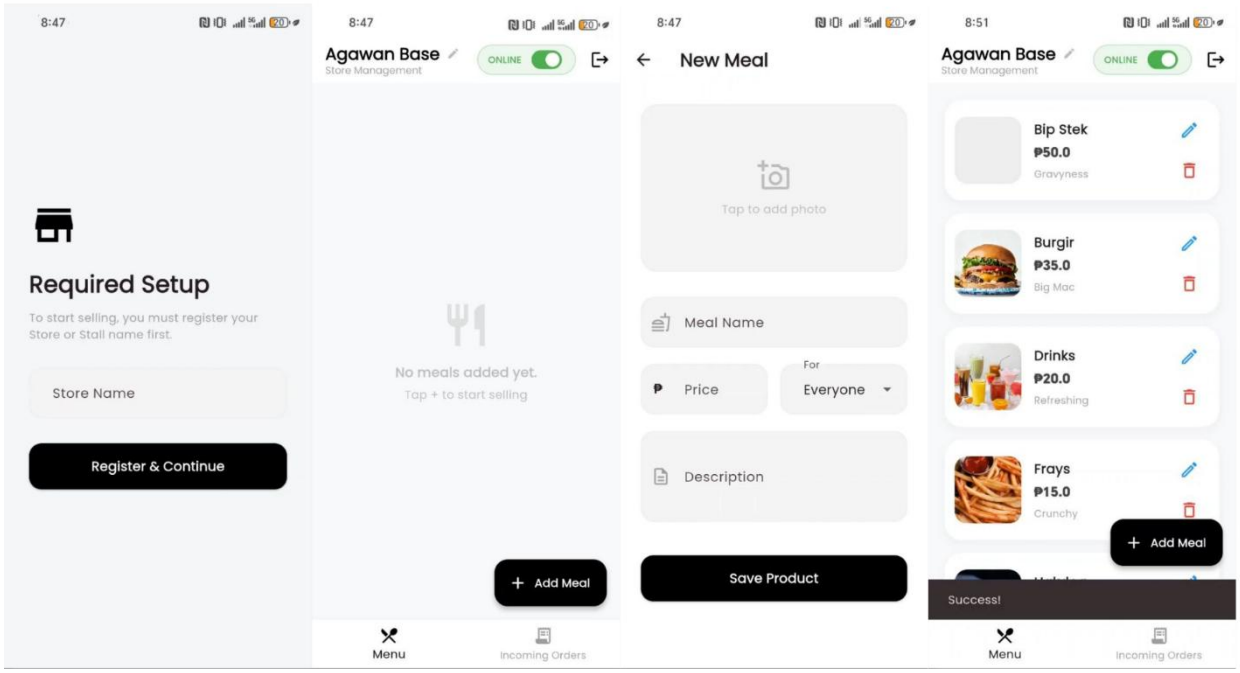
Ordering System / Add to Cart Feature



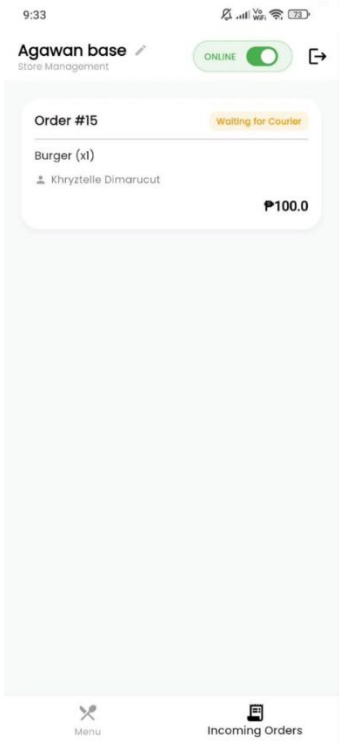
Finding Courier / Delivery in Progress / No Courier Found



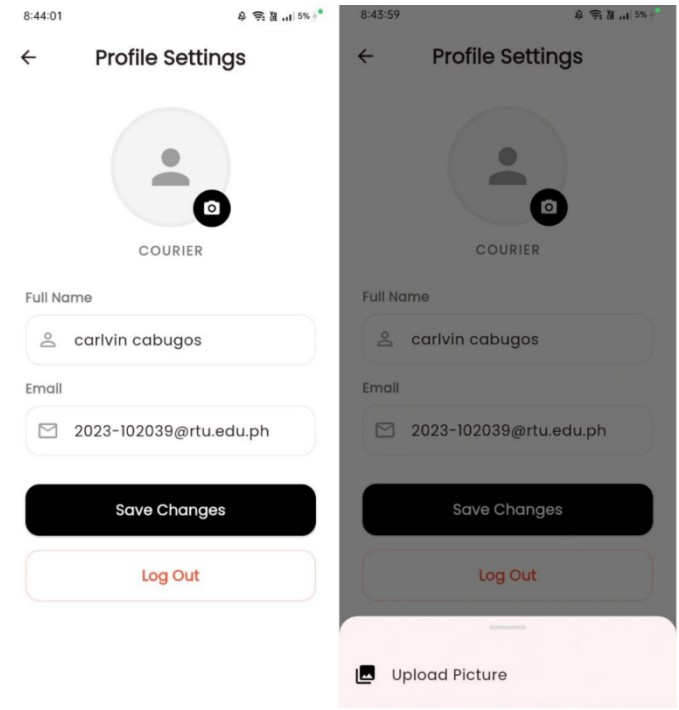
Staff



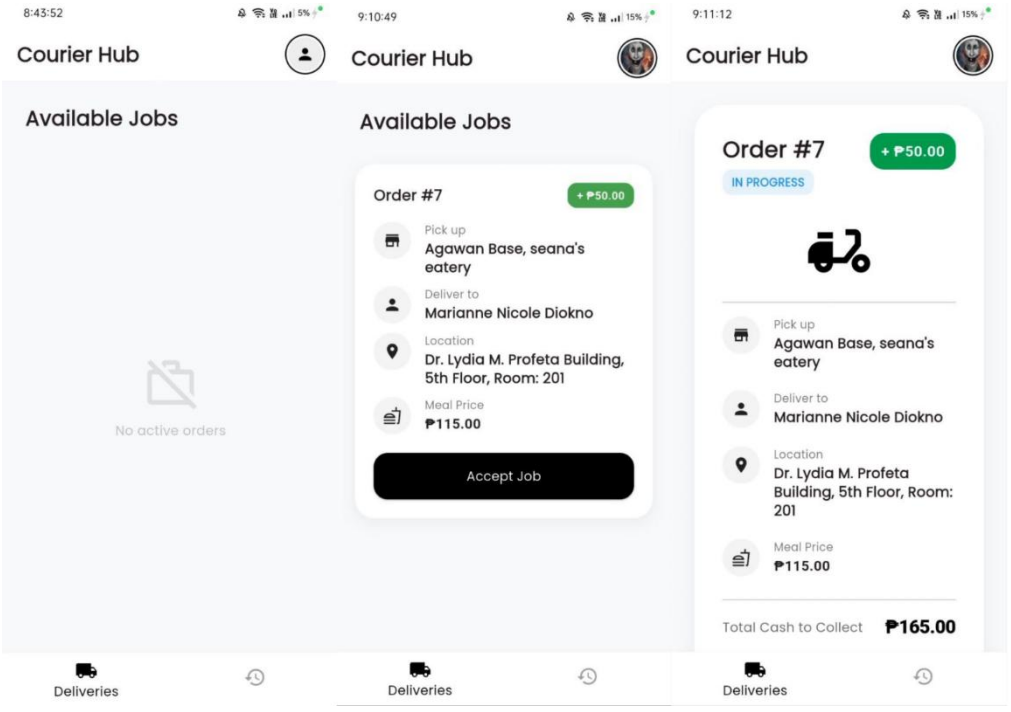
Staff (Incoming Order)



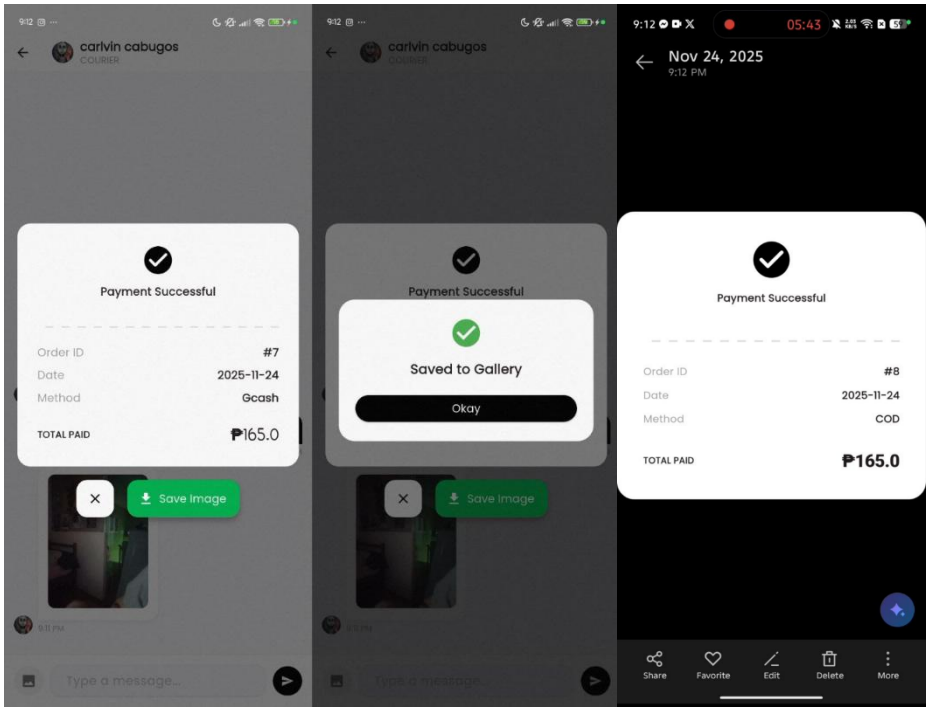
Courier / Delivery (Profile Settings)



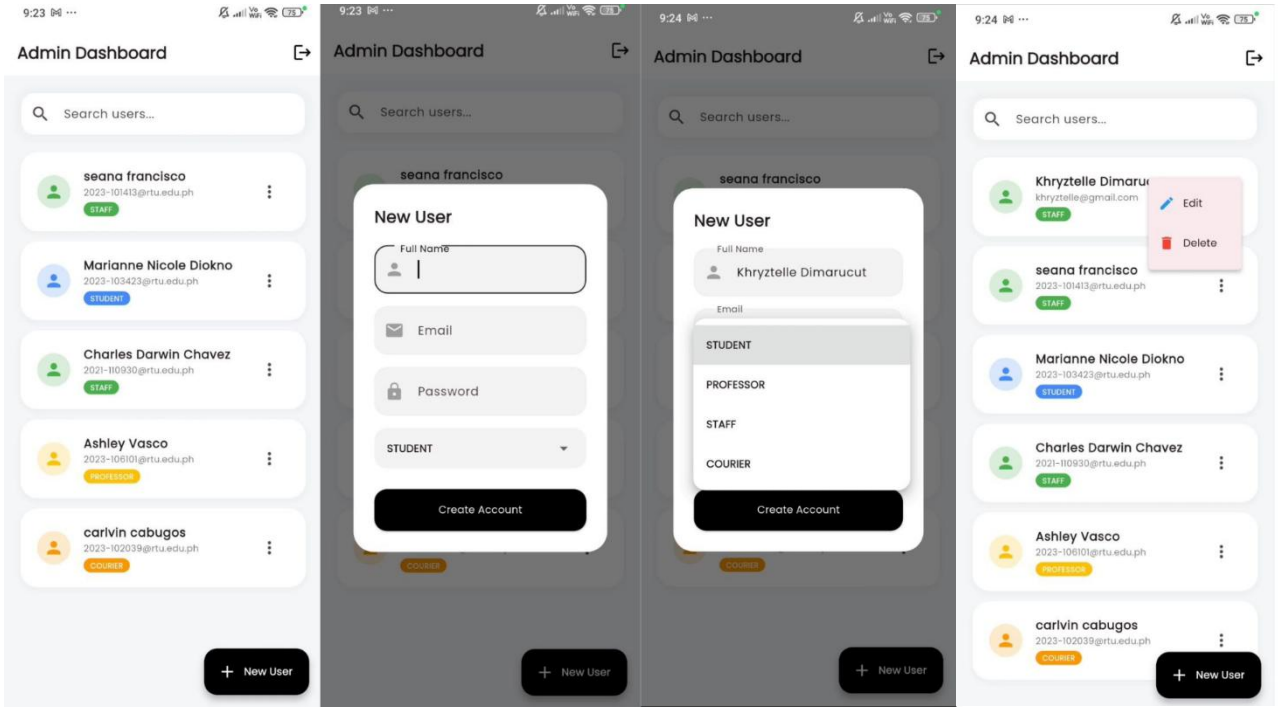
Courier Dashboard / In Progress Order

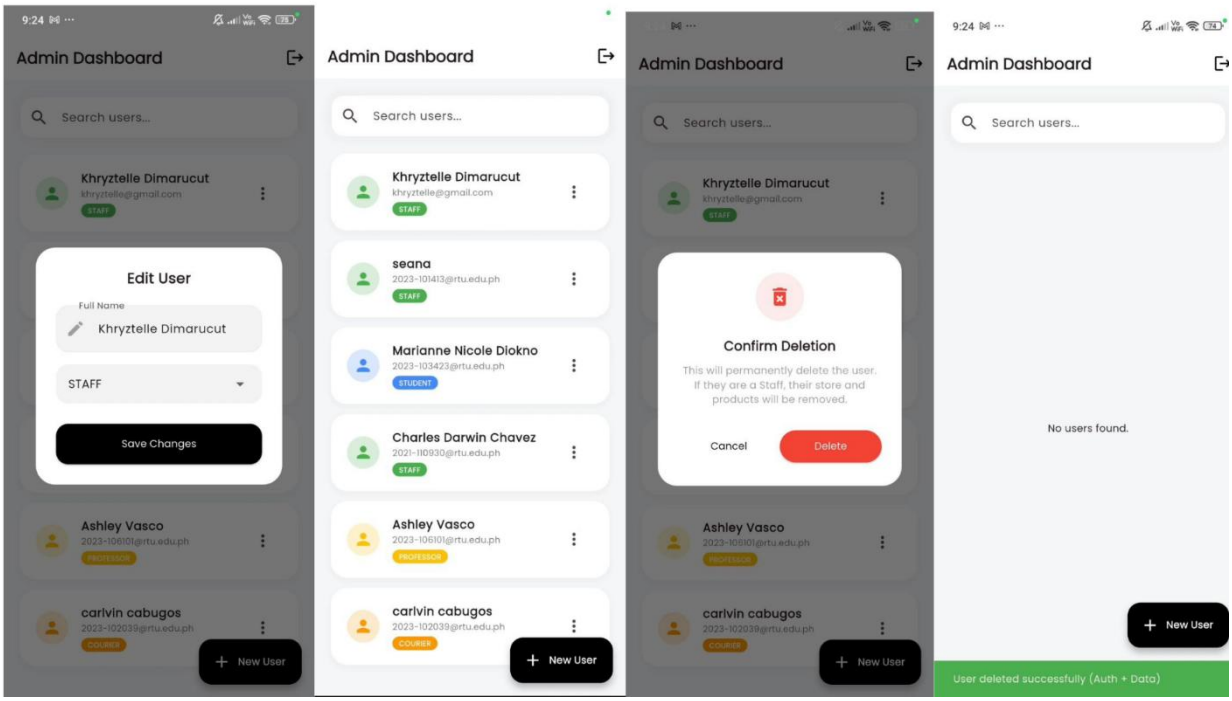


Message / Delivery Successful

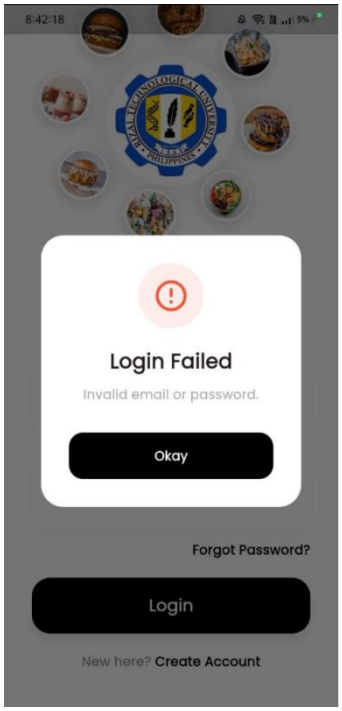


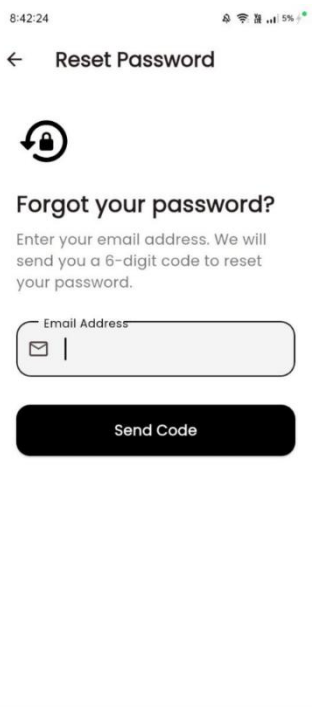
Admin Dashboard / Create, Read, Update and Delete

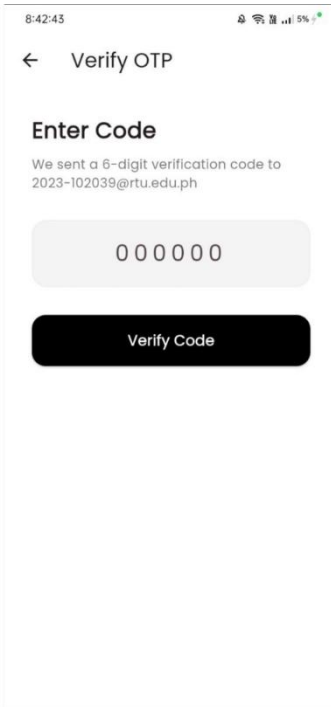


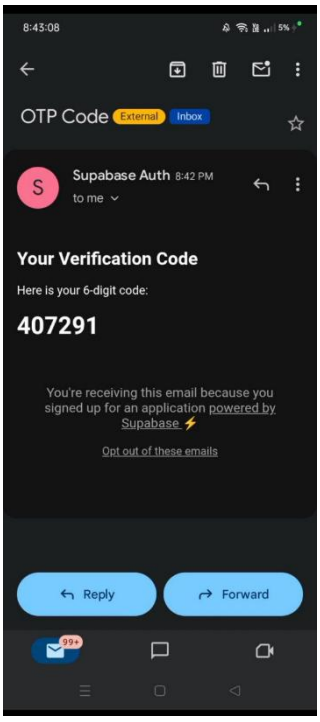


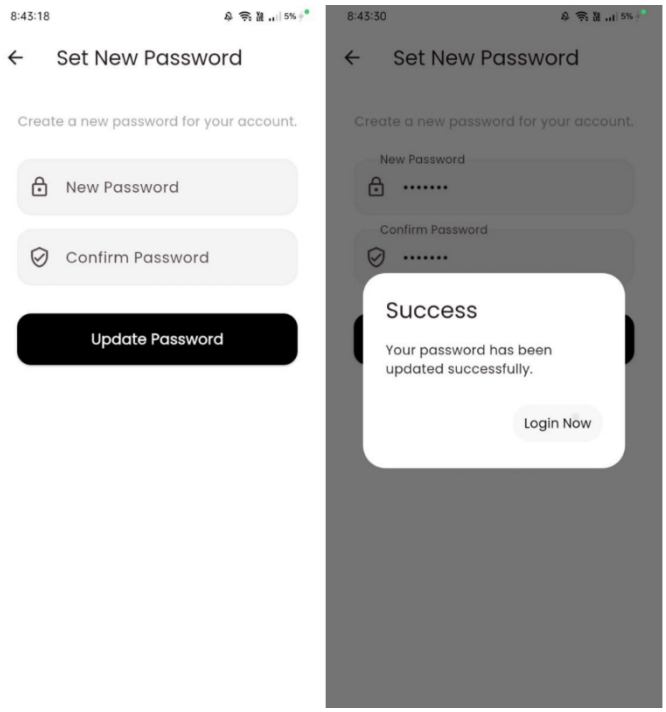
Forgot Password











VII. Conclusion and Recommendations

Conclusion

The development of the integrated POS and mobile-ordering platform was successfully accomplished through coordinated teamwork, persistent problem-solving, and systematic refinement despite multiple technical and logistical challenges. A key limitation identified is the absence of external cashless-payment gateway integration due to the lack of confirmed provider partnerships. Nonetheless, the completed prototype demonstrates strong scalability: its modular architecture, real-time notification system, and student-courier delivery workflow significantly streamline RTU canteen operations and reduce perceived waiting times. Empirical studies of campus dining technologies support these findings, indicating that mobile

ordering combined with POS integration decreases queue length and enhances service efficiency during peak periods. Furthermore, the centralized DBMS ensures atomic order-payment processing, immediate inventory synchronization, and stable performance under high-concurrency scenarios.

Recommendations

1. **Payment Integration** – Establish partnerships with digital-wallet and payment-gateway providers to fully support secure cashless transactions.
2. **User-Interface Refinement** – Implement responsive design patterns and accessibility guidelines to further improve usability and overall user experience.
3. **Database and Analytics Enhancement** – Introduce advanced inventory analytics, automated audit logs, and data-driven reporting features for better operational decision-making.
4. **Feature Expansion** – Develop additional functions such as scheduled delivery windows, smart-locker pick-up options, and customizable menu-filtering capabilities.

Addressing these areas will allow the system to evolve into a comprehensive campus-wide platform that enhances operational efficiency, user satisfaction, and organizational transparency.

VIII. References

Auti, P. et al. (2023). A Review on Smart Canteen Management System. ***IJRASET***. https://www.ijraset.com/research-paper/a-review-on-smart-canteen-management-system

Amazon Web Services. (n.d.). *What is a database management system (DBMS)?* AWS. https://aws.amazon.com/what-is/dbms/

Campus ID News. (2022). U. of Arizona, Grubhub create modern, convenient student dining experience. *Campus ID News.* https://www.campusidnews.com/u-of-arizona-grubhub-create-modern-convenient-student-dining-experience/

Dinesh, V. D., & Thakare, S. W. (2025). FoodGet: College campus food delivery system. *International Journal of Innovative Research in Computer and Communication Engineering, 13*(3). https://www.ijircce.com/

Fonngo, F., Jap, T., & Arisandi, D. (2020). *Web-based canteen payment and ordering system*. https://www.researchgate.net/publication/348106544_Web Based_Canteen_Payment_and_Ordering_System

Gao, T., & Su, X. (2022). The role of integrated POS systems in improving service operations: A systematic review. *Journal of Service Science and Management, 15*(2), 123–139. https://doi.org/10.4236/jssm.2022.152008

GoTeso. (2023). How Online Food Ordering Systems Are Changing the Restaurant Industry. https://www.goteso.com/blog/how-online-food-ordering-systems-are-changing-the-restaurant-industry

Hamid, J. N., Adnan, A., & Ekhsan, H. M. (2022). e-Runner: A mobile application for

campus food delivery service. *Journal of Computing Research and Innovation (JCRINN), 7*(2), 357–365. https://www.jcrinn.com/

Hashmato. (2024). Mobile POS Systems; Restaurant Benefits and Growth. https://hashmato.com/mobile-pos-systems-restaurant-benefits-growth

Hwang, J., & Kim, H. (2021). The impact of real-time order tracking on customer satisfaction in mobile ordering. *Service Industries Journal, 41*(9–10), 658–675. https://doi.org/10.1080/02642069.2020.1787995

Hudson, A. (2022, June 7). U. of Arizona, Grubhub create modern, convenient student dining experience. *Campus ID News.* https://www.campusidnews.com/u-of-arizona-grubhub-create-modern-convenient-student-dining-experience/

Mowreader, A. (2024, January 17). Survey: Students value choice in campus dining facilities. *Inside Higher Ed.* https://www.insidehighered.com/news/student-success/health-wellness/2024/01/17/what-college-students-want-their-dining-provider

Peblla. (2024). “Top 10 benefits of Using a Mobile POS System for Your Restaurant.” https://www.peblla.com/blog/top-10-benefits-of-using-a-mobile-pos-system-for-your-restaurant

Prajapati, K., & Shah, M. (2022). Application of queueing models in food service operations: Reducing waiting times through digital solutions. *Operations Research Perspectives, 9*, 100225. https://doi.org/10.1016/j.orp.2022.100225

Society for Hospitality and Foodservice Management. (2025, September 20). Food trends that are shaping campus dining. *SHFM.* https://shfm-online.org/food-trends-that-are-shaping-campus-dining/

Tan, C., & Lee, J. (2023). Evaluating the effectiveness of mobile ordering systems in managing student queues in university cafeterias. *Asian Journal of Technology and Management, 17*(1), 34–48. https://doi.org/10.7454/ajtm.v17i1.4221

Touchnet. (2024, April 9). 8 best practices for campus dining and meal plan management. *Touchnet Blog.* https://www.touchnet.com/trends/blog/2024/04/09/meal-plans-best-practices

Zhong, Y., Oh, S., & Moon, H. C. (2020). Service automation in the food industry: The impact of self-service kiosks on customer experience and efficiency. *Journal of Hospitality and Tourism Technology, 11*(3), 495–510. https://doi.org/10.1108/JHTT-01-2020-0010

IX. Appendices

SQL scripts

```
CREATE TABLE public.profiles (
  id UUID REFERENCES auth.users(id) ON DELETE CASCADE PRIMARY KEY,
  full_name TEXT,
  email TEXT,
  role TEXT DEFAULT 'student',
  avatar_url TEXT,
  store_name TEXT,
  is_open BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now())
NOT NULL
);

CREATE TABLE public.admins (
  id UUID REFERENCES auth.users(id) ON DELETE CASCADE PRIMARY KEY,
  email TEXT,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now())
NOT NULL
);

CREATE TABLE public.products (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
```

```
  owner_id UUID REFERENCES public.profiles(id) ON DELETE CASCADE NOT
NULL,
  name TEXT NOT NULL,
  price NUMERIC NOT NULL,
  description TEXT,
  image_url TEXT,
  target_audience TEXT DEFAULT 'all',
  created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now())
NOT NULL
);

CREATE TABLE public.orders (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  user_id UUID REFERENCES public.profiles(id) ON DELETE SET NULL,
  courier_id UUID REFERENCES public.profiles(id) ON DELETE SET NULL,

  customer_name TEXT,
  store_name TEXT,
  courier_name TEXT,
  items_summary TEXT,
  total_amount NUMERIC NOT NULL,
  status TEXT DEFAULT 'pending',
  type TEXT DEFAULT 'delivery',
  payment_method TEXT DEFAULT 'COD',
```

```
  delivery_location TEXT,

  created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now())
NOT NULL
);

CREATE TABLE public.order_items (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  order_id BIGINT REFERENCES public.orders(id) ON DELETE CASCADE NOT
NULL,
  product_id BIGINT REFERENCES public.products(id) ON DELETE SET NULL,

  product_name TEXT,
  customer_name TEXT,
  store_name TEXT,

  quantity INTEGER DEFAULT 1,
  price_at_purchase NUMERIC,

  created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now())
NOT NULL
);

CREATE TABLE public.chat_messages (
```

```
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  order_id BIGINT REFERENCES public.orders(id) ON DELETE CASCADE NOT NULL,
  sender_id UUID REFERENCES public.profiles(id) ON DELETE SET NULL,
  message TEXT,
  image_url TEXT,
  is_read BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT NULL
);

ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.products ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.order_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.chat_messages ENABLE ROW LEVEL SECURITY;

ALTER TABLE public.admins ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Public profiles are viewable by everyone."
ON public.profiles FOR SELECT USING (true);

CREATE POLICY "Users can update own profile."
ON public.profiles FOR UPDATE USING (auth.uid() = id);
```

```
CREATE POLICY "Admins read access"
ON public.admins FOR SELECT USING (auth.uid() = id);

CREATE POLICY "Products are viewable by everyone."
ON public.products FOR SELECT USING (true);

CREATE POLICY "Staff can insert own products."
ON public.products FOR INSERT WITH CHECK (auth.uid() = owner_id);

CREATE POLICY "Staff can update own products."
ON public.products FOR UPDATE USING (auth.uid() = owner_id);

CREATE POLICY "Staff can delete own products."
ON public.products FOR DELETE USING (auth.uid() = owner_id);

CREATE POLICY "Authenticated users can create orders."

ON public.orders FOR INSERT WITH CHECK (auth.role() = 'authenticated');

CREATE POLICY "Users can view relevant orders."
ON public.orders FOR SELECT USING (
  auth.uid() = user_id
  OR
  (auth.uid() = courier_id)
```

```
  OR
  (status = 'pending')
  OR
  EXISTS (
    SELECT 1 FROM public.profiles
    WHERE profiles.id = auth.uid()
    AND profiles.store_name = orders.store_name
  )
);

CREATE POLICY "Users can update relevant orders."
ON public.orders FOR UPDATE USING (
  auth.uid() = user_id
  OR
  auth.uid() = courier_id
  OR
  status = 'pending'
);

CREATE POLICY "Order items viewable by participants"
ON public.order_items FOR SELECT USING (
  EXISTS (
    SELECT 1 FROM public.orders
    WHERE orders.id = order_items.order_id
    AND (
```

```
      orders.user_id = auth.uid()
      OR orders.courier_id = auth.uid()
      OR EXISTS (
        SELECT 1 FROM public.profiles
        WHERE profiles.id = auth.uid()
        AND profiles.store_name = orders.store_name
      )
    )
  )
);

CREATE POLICY "Allow insert of order items"
ON public.order_items FOR INSERT WITH CHECK (auth.role() = 'authenticated');

CREATE POLICY "Chat participants can view messages"
ON public.chat_messages FOR SELECT USING (
  EXISTS (

    SELECT 1 FROM public.orders
    WHERE orders.id = chat_messages.order_id
    AND (orders.user_id = auth.uid() OR orders.courier_id = auth.uid())
  )
);
```

```
CREATE POLICY "Chat participants can insert messages"
ON public.chat_messages FOR INSERT WITH CHECK (
  EXISTS (
    SELECT 1 FROM public.orders
    WHERE orders.id = chat_messages.order_id
    AND (orders.user_id = auth.uid() OR orders.courier_id = auth.uid())
  )
);

CREATE POLICY "Mark messages as read"
ON public.chat_messages FOR UPDATE USING (
  EXISTS (
    SELECT 1 FROM public.orders
    WHERE orders.id = chat_messages.order_id
    AND (orders.user_id = auth.uid() OR orders.courier_id = auth.uid())
  )
);



CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO public.profiles (id, full_name, role, email)
  VALUES (
```

```
    new.id,
    new.raw_user_meta_data->>'full_name',
    COALESCE(new.raw_user_meta_data->>'role', 'student'),
    new.email
  );
  RETURN new;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE TRIGGER on_auth_user_created
  AFTER INSERT ON auth.users
  FOR EACH ROW EXECUTE PROCEDURE public.handle_new_user();

CREATE OR REPLACE FUNCTION public.delete_user_complete(target_user_id
UUID)
RETURNS VOID AS $$
BEGIN

  DELETE FROM auth.users WHERE id = target_user_id;

END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

INSERT INTO storage.buckets (id, name, public)
```

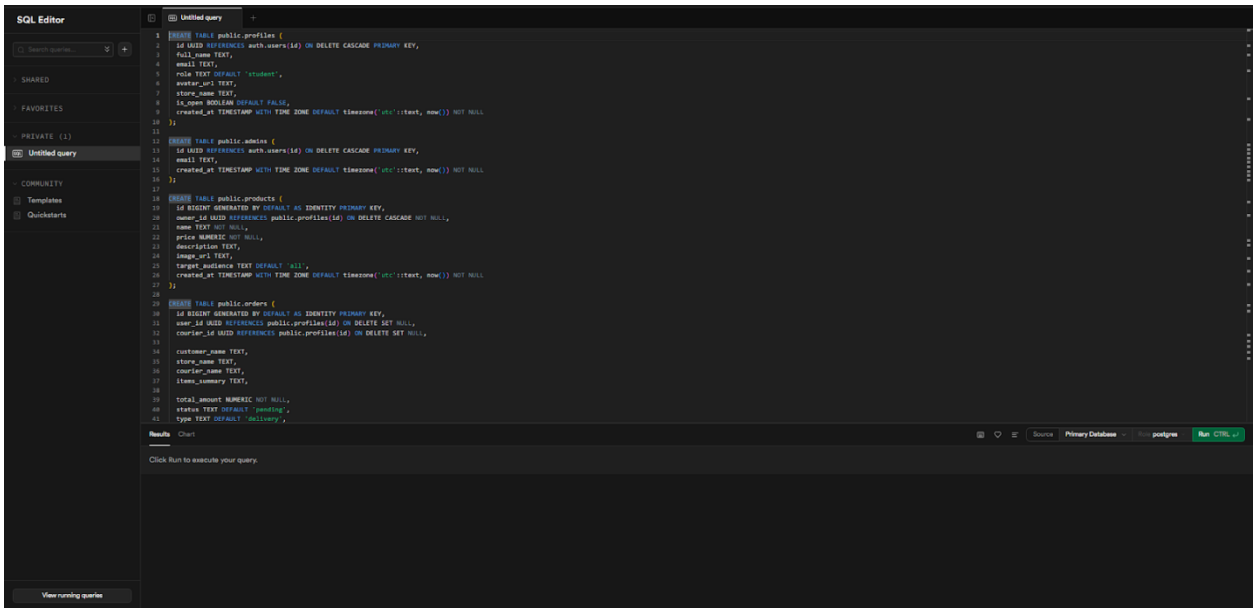
```
VALUES ('food-images', 'food-images', true)
ON CONFLICT (id) DO NOTHING;

INSERT INTO storage.buckets (id, name, public)
VALUES ('chat-attachments', 'chat-attachments', true)
ON CONFLICT (id) DO NOTHING;

CREATE POLICY "Public Access Food Images"
ON storage.objects FOR SELECT
USING ( bucket_id = 'food-images' );

CREATE POLICY "Authenticated Upload Food Images"
ON storage.objects FOR INSERT
WITH CHECK ( bucket_id = 'food-images' AND auth.role() = 'authenticated' );

CREATE POLICY "Owner Delete Food Images"
ON storage.objects FOR DELETE
USING ( bucket_id = 'food-images' AND auth.uid() = owner );

CREATE POLICY "Public Access Chat Images"

ON storage.objects FOR SELECT
USING ( bucket_id = 'chat-attachments' );
```

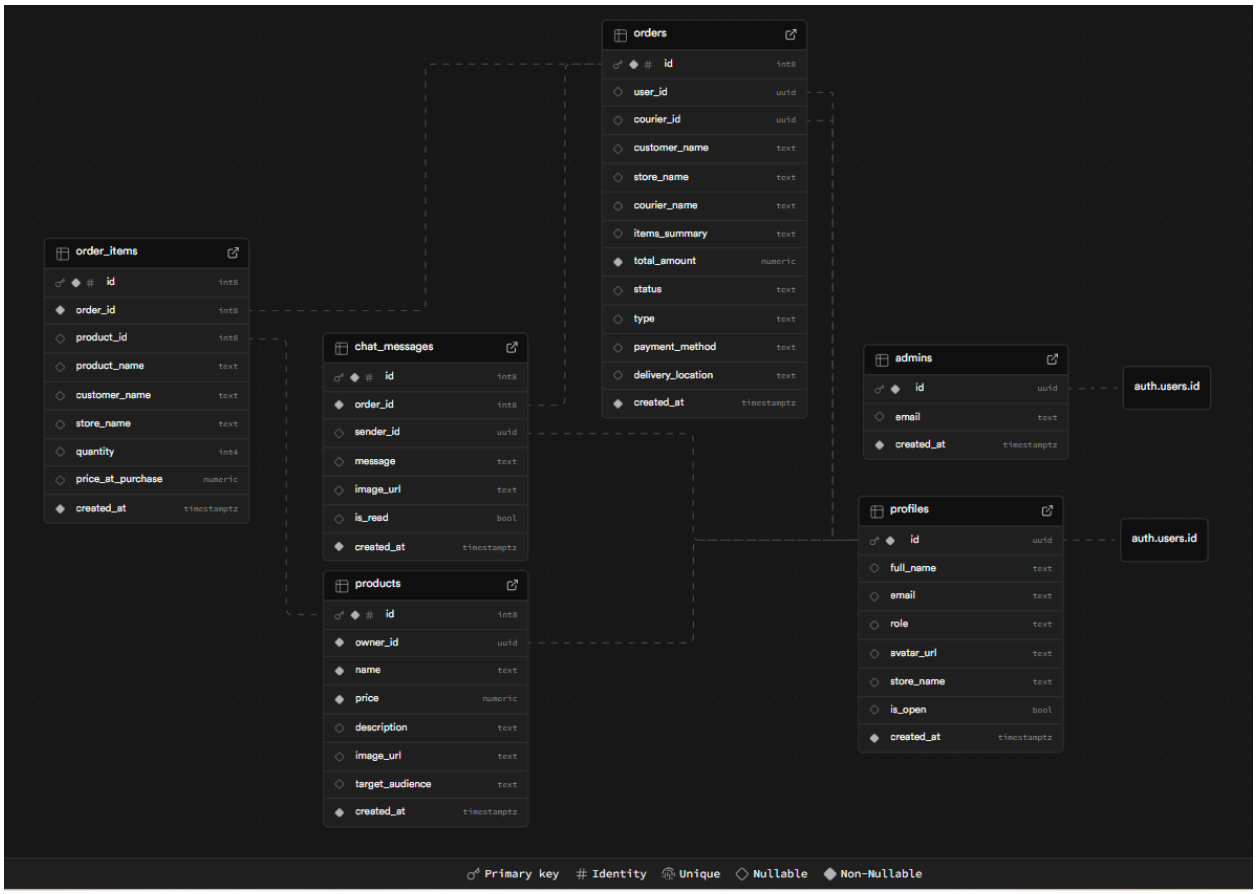
CREATE POLICY "Authenticated Upload Chat Images"

ON storage.objects FOR INSERT

WITH CHECK (bucket_id = 'chat-attachments' AND auth.role() = 'authenticated');

Additional diagrams or supporting documents





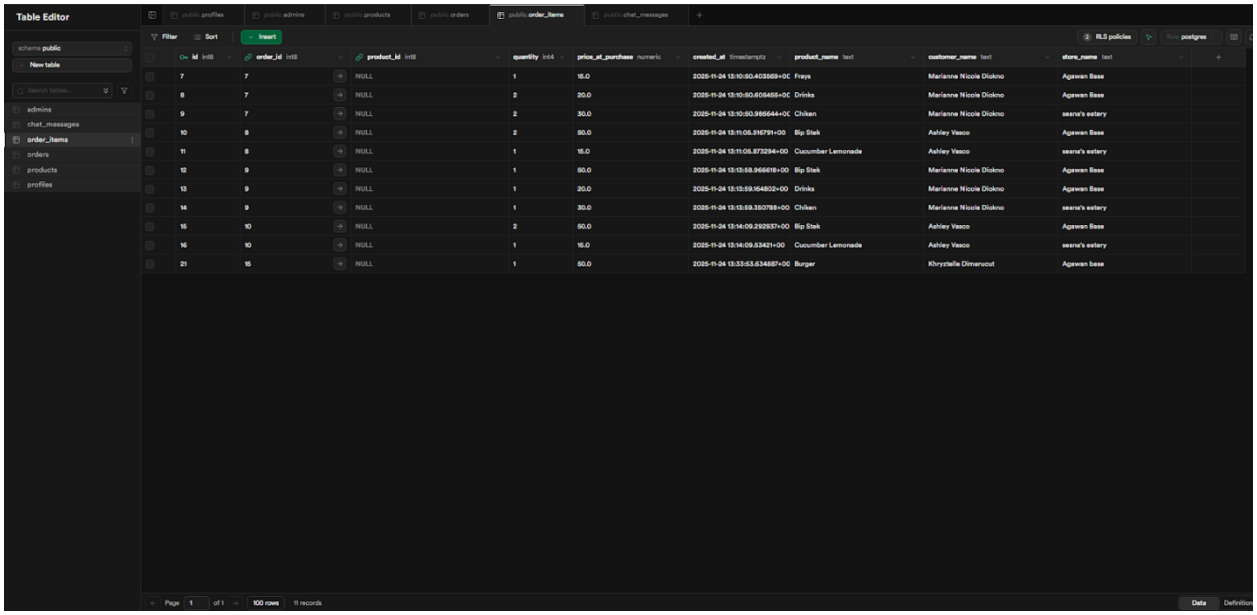


Table Editor
public.profiles
public.admins
public.products
public.orders
public.order_items
public.chat_messages
Filter
Sort
Insert
RLS policies
postgres
schema public
New table
admins
chat_messages
order_items
orders
products
profiles
id int8
order_id int8
product_id int8
quantity int4
price_at_purchase numeric
created_at timestamptz
product_name text
customer_name text
store_name text
7 7 NULL 1 15.0 2025-11-24 13:10:50.403569+00 Frays Marianne Nicole Diokno Agawan Base
8 7 NULL 2 20.0 2025-11-24 13:10:50.605455+00 Drinks Marianne Nicole Diokno Agawan Base
9 7 NULL 2 30.0 2025-11-24 13:10:50.986644+00 Chiken Marianne Nicole Diokno seana's eatery
10 8 NULL 2 50.0 2025-11-24 13:11:05.516791+00 Bip Stek Ashley Vasco Agawan Base
11 8 NULL 1 15.0 2025-11-24 13:11:05.873294+00 Cucumber Lemonade Ashley Vasco seana's eatery
12 9 NULL 1 50.0 2025-11-24 13:13:59.966618+00 Bip Stek Marianne Nicole Diokno Agawan Base
13 9 NULL 1 20.0 2025-11-24 13:13:59.164802+00 Drinks Marianne Nicole Diokno Agawan Base
14 9 NULL 1 30.0 2025-11-24 13:13:59.350788+00 Chiken Marianne Nicole Diokno seana's eatery
15 10 NULL 2 50.0 2025-11-24 13:14:09.292937+00 Bip Stek Ashley Vasco Agawan Base
16 10 NULL 1 15.0 2025-11-24 13:14:09.53421+00 Cucumber Lemonade Ashley Vasco seana's eatery
21 15 NULL 1 50.0 2025-11-24 13:33:53.634887+00 Burger Khryztelle Dimarucut Agawan base
Page 1 of 1
100 rows
11 records
Data
Definition

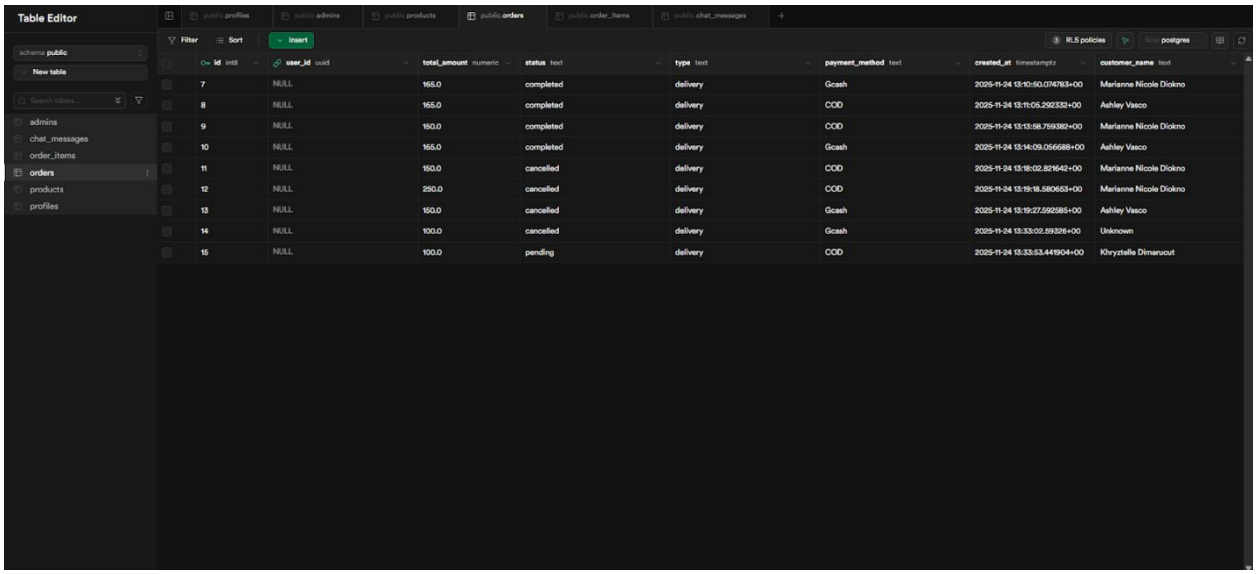


Table Editor
public.profiles
public.admins
public.products
public.orders
public.order_items
public.chat_messages
Filter
Sort
Insert
RLS policies
postgres
schema public
New table
admins
chat_messages
order_items
orders
products
profiles
id int8
user_id uuid
total_amount numeric
status text
type text
payment_method text
created_at timestamptz
customer_name text
7 NULL 165.0 completed delivery Gcash 2025-11-24 13:10:50.074783+00 Marianne Nicole Diokno
8 NULL 165.0 completed delivery COD 2025-11-24 13:11:05.292332+00 Ashley Vasco
9 NULL 150.0 completed delivery COD 2025-11-24 13:13:58.759382+00 Marianne Nicole Diokno
10 NULL 165.0 completed delivery Gcash 2025-11-24 13:14:09.056688+00 Ashley Vasco
11 NULL 150.0 cancelled delivery COD 2025-11-24 13:18:02.821642+00 Marianne Nicole Diokno
12 NULL 250.0 cancelled delivery COD 2025-11-24 13:19:18.580653+00 Marianne Nicole Diokno
13 NULL 150.0 cancelled delivery Gcash 2025-11-24 13:19:27.592585+00 Ashley Vasco
14 NULL 100.0 cancelled delivery Gcash 2025-11-24 13:33:02.59326+00 Unknown
15 NULL 100.0 pending delivery COD 2025-11-24 13:33:53.441904+00 Khryztelle Dimarucut

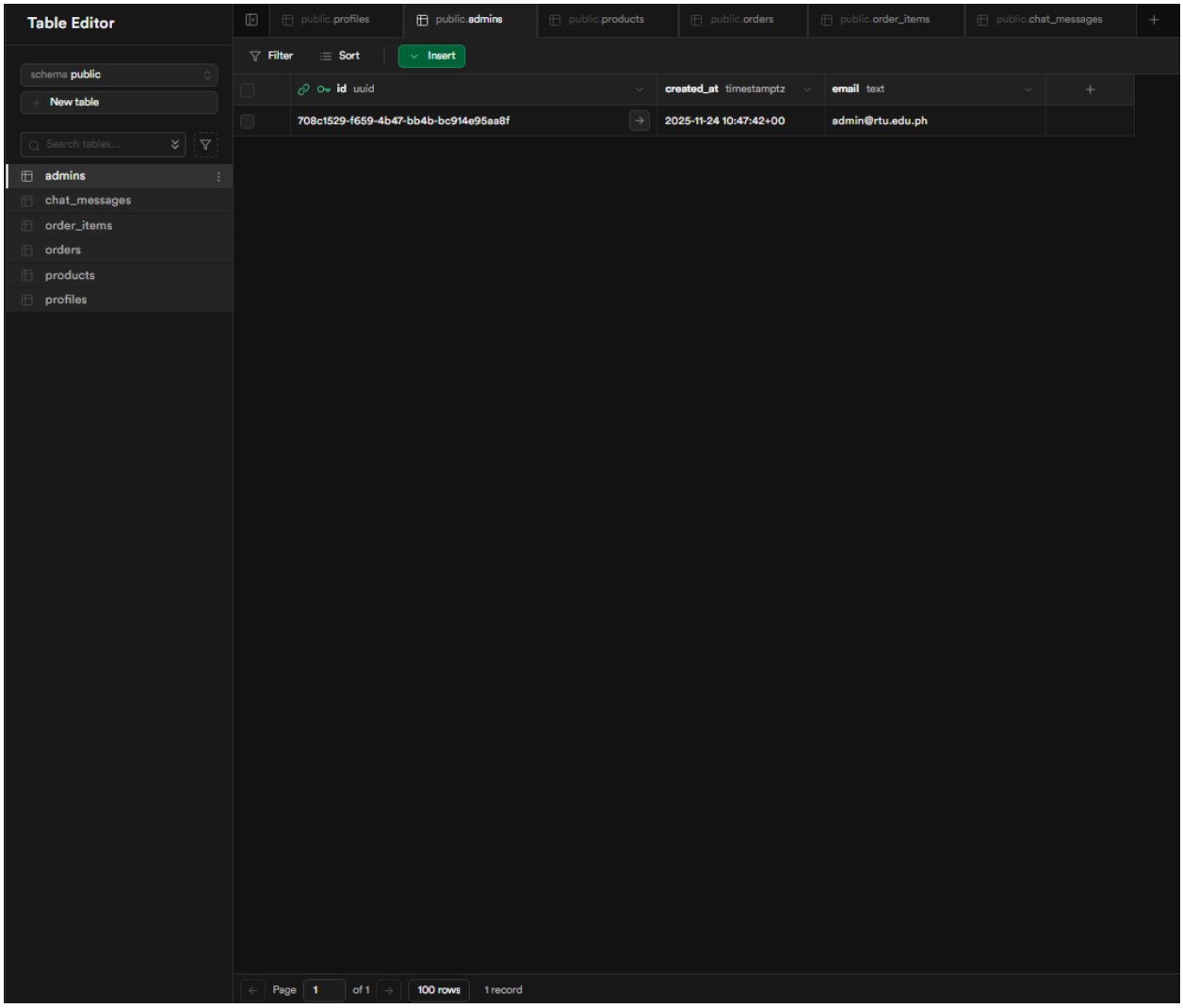
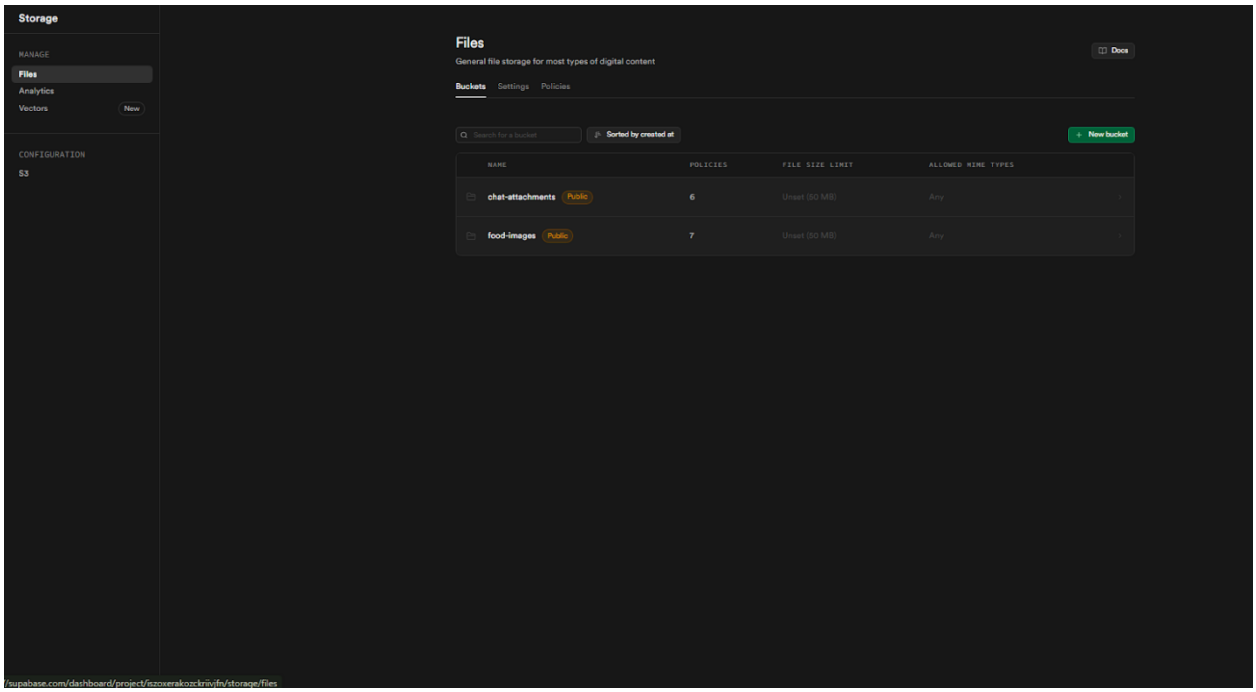
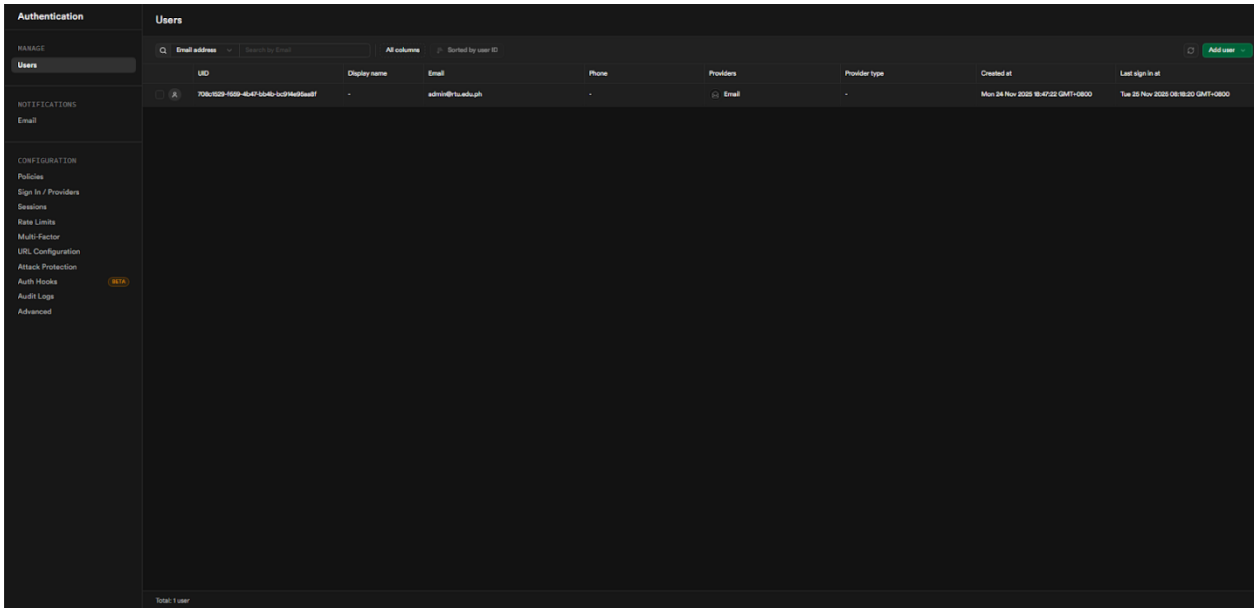


Table Editor
schema public
New table
Search tables...
admins
chat_messages
order_items
orders
products
profiles
public.profiles
public.admins
public.products
public.orders
public.order_items
public.chat_messages
Filter
Sort
Insert
id uuid
created_at timestamptz
email text
708c1529-f659-4b47-bb4b-bc914e95aa8f
2025-11-24 10:47:42+00
admin@rtu.edu.ph
Page 1 of 1
100 rows
1 record


Storage
MANAGE
Files
Analytics
Vectors
New
CONFIGURATION
S3
Files
General file storage for most types of digital content
Docs
Buckets
Settings
Policies
Search for a bucket
Sorted by created at
New bucket
NAME
POLICIES
FILE SIZE LIMIT
ALLOWED MIME TYPES
chat-attachments
Public
6
Unset (50 MB)
Any
food-images
Public
7
Unset (50 MB)
Any
/supabase.com/dashboard/project/iszoxerakozckriivjfn/storage/files


Authentication
MANAGE
Users
NOTIFICATIONS
Email
CONFIGURATION
Policies
Sign In / Providers
Sessions
Rate Limits
Multi-Factor
URL Configuration
Attack Protection
Auth Hooks
BETA
Audit Logs
Advanced
Users
Email address
Search by Email
All columns
Sorted by user ID
Add user
UID
Display name
Email
Phone
Providers
Provider type
Created at
Last sign in at
admin@tu.edu.ph
Email
Mon 24 Nov 2025 18:47:22 GMT+0800
Tue 25 Nov 2025 08:18:20 GMT+0800
Total: 1 user